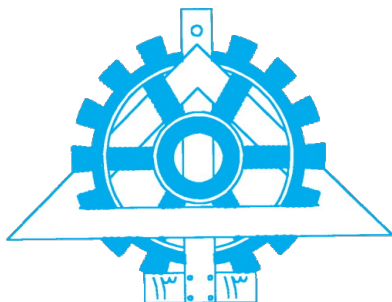




دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر
گروه مهندسی کامپیوتر



کاهش مدل معنایی ربکا با استفاده از رابطه شبیه‌سازی دوسویه ضعیف زمان‌دار

پروژه کارشناسی مهندسی کامپیوتر گرایش نرم‌افزار

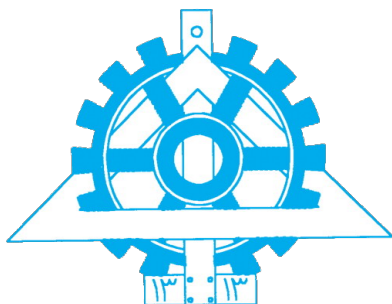
امیرپارسا موبد

استاد راهنما

دکتر فاطمه قاسمی اصفهانی

شهریور ۱۴۰۵





دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر
گروه مهندسی کامپیوتر



کاهش مدل معنایی ربکا با استفاده از رابطه شبیه‌سازی دوسویه ضعیف زمان‌دار

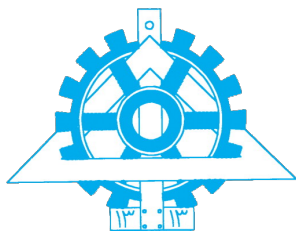
پروژه کارشناسی مهندسی کامپیوتر گرایش نرم‌افزار

امیرپارسا موبد

استاد راهنما

دکتر فاطمه قاسمی اصفهانی

شهریور ۱۴۰۵



دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر



گواهی دفاع از پروژه کارشناسی

هیأت داوران پروژه کارشناسی آقای / خانم امیرپارسا موبد به شماره دانشجویی ۸۱۰۱۰۰۲۷۱ در رشته مهندسی کامپیوتر - گرایش نرم افزار را در تاریخ با عنوان «کاهش مدل معنایی ربکا با استفاده از رابطه شبیه سازی دوسویه ضعیف زمان دار»

به عدد	به حروف

با نمره نهایی

و درجه

ارزیابی کرد.

ردیف	مشخصات هیأت داوران	نام و نام خانوادگی	مرتبه دانشگاهی	دانشگاه یا مؤسسه	امضا
۱	استاد راهنما	دکتر فاطمه قاسمی اصفهانی	استادیار	دانشگاه تهران	
۲	استاد داور داخلی	دکتر رامتین خسروی		دانشگاه تهران	

نام و نام خانوادگی معاون آموزشی و تحصیلات

تکمیلی پردیس دانشکده‌های فنی:

تاریخ و امضا:

نام و نام خانوادگی معاون تحصیلات تکمیلی و

پژوهشی دانشکده / گروه:

تاریخ و امضا:

تعهدنامه اصالت اثر

باسمه تعالی

اینجانب امیرپارسا موبد تأیید می‌کنم که مطالب مندرج در این پروژه حاصل کار پژوهشی اینجانب است و به دستاوردهای پژوهشی دیگران که در این نوشته از آنها استفاده شده است مطابق مقررات ارجاع گردیده است. این پروژه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتری ارائه نشده است.

نام و نام خانوادگی دانشجو: امیرپارسا موبد

تاریخ و امضای دانشجو:

کلیه حقوق مادی و معنوی این اثر
متعلق به دانشگاه تهران است.

چکیده

وارسی مدل با پیمایش فضای حالت یک سامانه، ویژگی‌های مورد انتظار را به صورت خودکار بررسی می‌کند. در مدل‌های هم‌روند و زمان‌دار، ترکیب ترتیب اجرای مؤلفه‌ها، صف پیام‌ها و محدودیت‌های زمانی به انفجار فضای حالت می‌انجامد و حاصل آن، یک سامانه انتقال زمان‌دار بسیار بزرگ است. این پایان‌نامه به دنبال جلوگیری از این انفجار در هنگام تولید فضای حالت نیست؛ مسئله آن، کوچک کردن همان سامانه بزرگ پس از تولید آن است، تا تحلیل کردن، مشاهده کردن و مقایسه آن با مدلی دیگر عملی شود.

برای این کار، ابزاری ساخته شده است که خروجی فضای حالت زمان‌دار افرا را می‌خواند، تعامل‌هایی را که کاربر قابل مشاهده اعلام نکرده است پنهان می‌کند، و حالت‌هایی را که از نظر رفتار قابل مشاهده و زمان سپری شده تمایزپذیر نیستند در یک رده قرار می‌دهد؛ سپس از این رده‌ها مدل خارج‌قسمتی کوچک‌تری می‌سازد و رابطه آن با ورودی را با واریسی‌گری مستقل بررسی می‌کند. معیار هم‌ارزی، شبیه‌سازی دوسویه ضعیف زمان‌دار است و ساختار الگوریتم از پالایش افزایشی گرفته شده که برای محاسبه شبیه‌سازی دوسویه انشعابی روی سامانه‌های انتقال برچسب‌دار ارائه شده است.

زنجیره ابزار ربکا و افرا فضای حالت زمان‌دار را تولید و نمایش می‌دهد، اما پیش از این ابزاری در دسترس کاربران آن نبود که همین فضای حالت را نسبت به مجموعه‌ای از تعامل‌های قابل مشاهده کاهش دهد؛ فراهم کردن چنین ابزاری، سهم اصلی این کار است. ارزیابی روی نمونه‌های موجود و روی فضای حالت‌های تولیدشده از مدل‌های واقعی ربکا نشان داد که این روش می‌تواند ضمن حفظ معیار رفتاری انتخاب‌شده، اندازه مدل را کاهش دهد و مدل‌های متفاوتی را که رفتار قابل مشاهده یکسانی دارند به یک مدل کاهش یافته برساند.

واژگان کلیدی ربکا، افرا، شبیه‌سازی دوسویه ضعیف زمان‌دار، روش‌های صوری، واریسی مدل

فهرست مطالب

ث فهرست تصاویر

ج فهرست جداول

چ فهرست الگوریتم‌ها

۱ فصل ۱: مقدمه

۱ ۱.۱ نرم‌افزار در تعامل با جهان

۲ ۲.۱ هزینه یک فرض زمانی نادرست

۳ ۳.۱ از آزمون تا درستی‌یابی صوری

۴ ۴.۱ انفجار فضای حالت و جایگاه این پروژه

۵ ۵.۱ مقایسه پیاده‌سازی و مشخصه

۵ ۶.۱ ایده الگوریتم

۶ ۷.۱ بستر ربکا و افرا

۷ ۸.۱ هدف‌ها و پرسش‌های پروژه

۸ ۹.۱ دامنه و دستاوردهای پروژه

۸ ۱۰.۱ روش کلی ارزیابی

۹ ۱۱.۱ ساختار پایان‌نامه

۱۱ فصل ۲: مبانی نظری و تعاریف

۱۲ ۱.۲ از آزمون نرم‌افزار تا درستی‌یابی صوری

۲.۲	وارسی مدل	۱۳
۳.۲	سیستم‌های واکنشی و بلادرنگ	۱۴
۴.۲	مدل کنشگر، ربکا و افرا	۱۴
۵.۲	سامانه انتقال برچسب‌دار	۱۶
۶.۲	سامانه انتقال زمان‌دار	۱۷
۷.۲	هم‌ارزی‌های رفتاری روی سامانه انتقال برچسب‌دار	۱۹
۸.۲	شبیه‌سازی دوسویه زمان‌دار	۲۱
۹.۲	زمان گسسته و زمان چگال	۲۳
۱۰.۲	هم‌ارزی، افراز و کاهش مدل	۲۳
۲۵	فصل ۳: روش طراحی و پیاده‌سازی	
۱.۳	مقدمه	۲۵
۲.۳	صورت مسئله، ورودی و خروجی	۲۶
۳.۳	جریان پردازش	۲۶
۴.۳	خواندن خروجی افرا	۲۷
۱.۴.۳	قرارداد فایل TTS	۲۷
۲.۴.۳	مرز منبع داده	۲۸
۵.۳	انتخاب مشاهده‌ها و پیش‌پردازش	۲۸
۶.۳	چرا یال‌های تأخیر به گام‌های یک‌واحدی شکسته می‌شوند	۲۹
۱.۶.۳	مسئله: حالت‌هایی که در گراف ذخیره‌شده شریکی ندارند	۲۹
۲.۶.۳	راه‌حل: استفاده از ویژگی پیوستگی	۳۰
۳.۶.۳	هزینه و سقف کنترلی	۳۱
۴.۶.۳	دو حالت اجرایی و تفاوت معنایی آن‌ها	۳۱
۷.۳	ساخت رابطه‌گذار ضعیف	۳۲
۸.۳	ایده مبنا: پالایش افراز برای شبیه‌سازی دوسویه انشعابی	۳۳
۱.۸.۳	مسئله درشت‌ترین افراز	۳۳

۳۳	آنچه گروته و واندراخر افزودند	۲.۸.۳
۳۴	آنچه در این پروژه تغییر کرده است	۳.۸.۳
۳۵	پالایش افراز	۹.۳
۳۹	ساخت مدل خارج قسمتی و فشرده سازی زمان	۱۰.۳
۳۹	وارسی مستقل خروجی	۱۱.۳
۴۰	معماری نرم افزار و رابط اجرا	۱۲.۳
۴۱	مصنوعات خروجی	۱۳.۳
۴۱	طرح اعتبارسنجی و تکرار اجرا	۱۴.۳
۴۲	جمع بندی	۱۵.۳
۴۳	نتایج و اعتبارسنجی	فصل ۴:
۴۳	مقدمه	۱.۴
۴۴	ورودی های ارزیابی	۲.۴
۴۴	شرایط اجرا و سنجه ها	۳.۴
۴۵	نتایج کاهش	۴.۴
۴۶	مطالعه موردی خانه هوشمند	۵.۴
۴۸	مطالعه موردی مدل های ربکا	۶.۴
۴۸	خانواده mood: تأخیر یکپارچه در برابر تأخیر شکسته	۱.۶.۴
۵۳	خانواده teacher: متغیر حالتی که رفتار قابل مشاهده را تغییر نمی دهد	۲.۶.۴
۵۶	خانواده chain: زنجیره پیام در برابر بیان فشرده	۳.۶.۴
۵۶	جمع بندی عددی	۴.۶.۴
۵۹	حساسیت به معنای زمان و مجموعه مشاهده ها	۷.۴
۶۰	نتایج وارسی رابطه خروجی	۸.۴
۶۱	پوشش آزمون ها	۹.۴
۶۲	زمان اجرا و حافظه	۱۰.۴
۶۲	جمع بندی	۱۱.۴

فصل ۵:	بحث و نتیجه‌گیری	۶۳
۱.۵	مرور مسیر پروژه	۶۳
۲.۵	پاسخ به پرسش‌های پروژه	۶۴
۱.۲.۵	خواندن و مستقل کردن خروجی افرا	۶۴
۲.۲.۵	کاهش همراه با واریسی مستقل	۶۴
۳.۲.۵	اثر معنای زمان و اندازه کاهش	۶۵
۳.۵	تفسیر تصمیم‌های طراحی	۶۵
۴.۵	جایگاه دستاورد	۶۶
۵.۵	محدودیت‌ها	۶۷
۶.۵	کارهای آینده	۶۷
۷.۵	جمع‌بندی نهایی	۶۹
فصل آ:	مدل‌های ربکای به‌کاررفته در ارزیابی	۷۱
۱.آ	خانواده mood	۷۱
۲.آ	خانواده teacher	۷۱
۳.آ	خانواده chain	۷۴
کتاب‌نامه		۷۷
واژه‌نامه فارسی به انگلیسی		سوم
واژه‌نامه انگلیسی به فارسی		پنجم

فهرست تصاویر

۱.۳	جریان پردازش در ابزار کاهش فضای حالت زمان‌دار.	۲۷
۱.۴	فضای حالت خانه هوشمند پیش و پس از کاهش ضعیف زمان‌دار.	۴۷
۲.۴	فضای حالت تولیدشده برای mood1؛ ۱۰ حالت و ۱۲ گذار. نمودار با StateSpaceTransformer	
۵۱	افرا تولید شده است.	
۳.۴	فضای حالت تولیدشده برای mood2؛ ۶ حالت و ۸ گذار.	۵۲
۴.۴	مدل خارج‌قسمتی مشترک mood1 و mood2؛ ۵ حالت و ۶ گذار.	۵۳
۵.۴	فضای حالت تولیدشده برای teacher1 و teacher3؛ ۱۹ حالت و ۱۹ گذار.	۵۴
۶.۴	فضای حالت تولیدشده برای teacher2؛ ۷ حالت و ۷ گذار.	۵۵
۷.۴	مدل خارج‌قسمتی مشترک هر سه مدل teacher؛ ۲ حالت و ۲ گذار.	۵۵
۸.۴	فضای حالت تولیدشده برای chain1؛ ۲۳ حالت و ۲۵ گذار.	۵۷
۹.۴	فضای حالت تولیدشده برای chain2؛ ۱۰ حالت و ۱۲ گذار.	۵۸
۱۰.۴	مدل خارج‌قسمتی مشترک chain1 و chain2؛ ۷ حالت و ۸ گذار.	۵۹

فهرست جداول

۱.۳	بخش مورد استفاده از قرارداد خروجی فضای حالت افرا	۲۷
۲.۳	مصنوعات تولیدشده برای هر اجرای کاهش	۴۱
۱.۴	ورودی‌ها و هدف هر مورد ارزیابی	۴۴
۲.۴	شمار حالت‌ها، گذارها و نتیجه کاهش	۴۵
۳.۴	نسبت کاهش حالت‌ها و گذارها	۴۶
۴.۴	کاهش فضای حالت‌های تولیدشده از مدل‌های ربکا	۵۶
۵.۴	حساسیت نتیجه به تفسیر تأخیر	۵۹
۶.۴	خلاصه واریسی مستقل مدل‌های کاهش یافته	۶۰
۷.۴	تعداد آزمون‌ها بر حسب هدف	۶۱
۸.۴	زمان و بیشینه حافظه در اجرای ثبت شده	۶۲

فهرست الگوریتم‌ها

۳۶		Tau Closure	۱.۳
۳۷		Weak Successors for One Label	۲.۳
۳۷		Weak Transition Relation	۳.۳
۳۸		Stable Partition Refinement	۴.۳

فصل ۱

مقدمه

۱.۱ نرم افزار در تعامل با جهان

بخش مهمی از نرم افزارهای امروز فقط ورودی ثابتی را دریافت نمی کنند تا پس از محاسبه خاتمه یابند. یک کنترل کننده آسانسور باید در تمام مدت کار ساختمان به درخواست طبقه، بازشدن در، مانع میان در و وضعیت موتور واکنش نشان دهد. یک خدمت دهنده شبکه باید هم زمان پیام چند کاربر، پایان مهلت ها و قطع ارتباط را مدیریت کند. چراغ راهنمایی، کنترل کننده صنعتی و سامانه پایش بیمار نیز پیوسته داده ای از محیط می گیرند و رفتار بعدی خود را بر اساس حالت جاری انتخاب می کنند. این دسته از نرم افزارها نمونه هایی از سیستم های واکنشی^۱ هستند؛ یعنی سامانه هایی که معنای رفتارشان در یک تعامل ادامه دار با محیط شکل می گیرد [۲۸، ۲۹].

در یک برنامه ترتیبی کوچک، یافتن خروجی نادرست برای یک ورودی معمولاً مسیر نسبتاً روشنی برای رفع خطا فراهم می کند. در سامانه واکنشی، نتیجه به تاریخچه رویدادها نیز وابسته است. ممکن است دو پیام جداگانه درست پردازش شوند، اما رسیدن آن ها با ترتیبی خاص به بن بست بینجامد؛ یا خطا تنها پس از تکرار طولانی یک چرخه آشکار شود. اگر چند مؤلفه مستقل و ناهمگام باشند، تعداد ترتیب های ممکن اجرای همان چند رویداد نیز به سرعت افزایش می یابد. بنابراین مسئله فقط درست بودن یک تابع نیست، بلکه باید رفتار همه اجزای مرتبط بررسی شود.

زمان در برخی از این سامانه ها جزئی از تعریف درستی است. در پخش صوت، دیررسیدن یک بسته شاید

¹Reactive System

تنها به افت کیفیت منجر شود؛ اما در ترمز خودرو، تشخیص برخورد یا رهگیری یک هدف، پاسخ درست پس از پایان مهلت دیگر پاسخ قابل قبول نیست. چنین سامانه‌ای سیستم بلادرنگ^۲ نامیده می‌شود. صفت «بلادرنگ» الزاماً به معنای سریع بودن مطلق نیست؛ معنای دقیق‌تر آن این است که رویداد یا پاسخ باید در بازه زمانی تعیین شده رخ دهد. برای نمونه، کنترل دمای یک اتاق می‌تواند مهلتی چندثانیه‌ای داشته باشد، درحالی‌که کنترل یک فرایند فیزیکی سریع به مهلتی بسیار کوتاه‌تر نیاز دارد. آنچه اهمیت دارد پیوند رفتار منطقی و محدودیت زمانی است [۱، ۲].

۲.۱ هزینه یک فرض زمانی نادرست

رخداد سامانه دفاع موشکی پاتریوت در طهران نمونه‌ای روشن از اهمیت همین پیوند است. در ۲۵ فوریه ۱۹۹۱، سامانه نتوانست یک موشک اسکاد ورودی را رهگیری کند و برخورد موشک با محل استقرار نیروها به کشته شدن ۲۸ نفر انجامید. بررسی دفتر حسابرسی عمومی ایالات متحده نشان داد محاسبه زمان در رایانه کنترل تسلیحات بر اثر تبدیل عددی دقت از دست می‌داد. خطای کوچک هر محاسبه با طول مدت کار سامانه انباشته می‌شد؛ سامانه در زمان حادثه بیش از ۱۰۰ ساعت پیوسته کار کرده بود و در نتیجه، محدوده جست‌وجوی هدف را در محل نادرستی محاسبه می‌کرد [۳۱].

این رخداد تنها یک «خطای گردکردن» جدا از طراحی سامانه نبود. پاتریوت در ابتدا برای استقرارهای کوتاه طراحی شده بود و استفاده پیوسته طولانی در فرض‌های اولیه آن قرار نداشت. رفتار نرم‌افزار برای چند ساعت می‌توانست پذیرفتنی باشد، اما همان محاسبه در افق زمانی طولانی ویژگی لازم برای رهگیری را نقض می‌کرد. یک مشخصه صوری که کران خطای زمان یا قراردادن هدف در پنجره رهگیری را برای کل مدت مجاز کار بیان کند، این فرض پنهان را به یک تعهد قابل بررسی تبدیل می‌کند. این گزاره به معنای آن نیست که هر استفاده از روش صوری حتماً حادثه را پیش‌بینی می‌کرد؛ اگر محدوده کار یا ویژگی مورد انتظار اشتباه مشخص شود، اثبات نیز همان فرض اشتباه را معتبر می‌گیرد. ارزش روش صوری در واداشتن طراح به بیان دقیق چنین فرض‌هایی و بررسی همه حالت‌های مدل‌شده است.

شکست نخستین پرواز آریان ۵ در سال ۱۹۹۶ جنبه دیگری از مسئله را نشان می‌دهد. موشک حدود ۴۰ ثانیه پس از آغاز پرواز از مسیر خارج شد و از بین رفت. گزارش رسمی آژانس فضایی اروپا علت را خطاهای

^۲Real-Time System

مشخصه و طراحی در نرم‌افزار سامانه مرجع اینرسی دانست؛ دو واحد افزونه نیز به دلیل اجرای نرم‌افزار یکسان تقریباً هم‌زمان از کار افتادند. آزمون‌های انجام‌شده مسیر واقعی آریان ۵ را به اندازه کافی بازنمایی نکرده بودند و در نتیجه پیش شرط نامعتبر تبدیل عددی کشف نشد [۸]. این مثال یادآوری می‌کند که افزونگی سخت‌افزاری در برابر خطای مشترک طراحی محافظت کافی ایجاد نمی‌کند و پوشش آزمون نیز به سناریوهای انتخاب‌شده محدود است.

۳.۱ از آزمون تا درستی‌یابی صوری

آزمون نرم‌افزار همچنان بخش ضروری فرایند مهندسی است: اجرای واقعی می‌تواند خطای پیاده‌سازی، ناسازگاری محیط و مسئله‌ای را آشکار کند که در مدل انتزاعی حضور ندارد. محدودیت بنیادی آزمون این است که هر مورد آزمون تنها یک یا چند اجرای مشخص را مشاهده می‌کند. در سامانه‌های هم‌رند و زمان‌دار، فضای ترتیب پیام‌ها، انتخاب‌ها و مقدارهای زمانی ممکن است چنان بزرگ باشد که پیمایش دستی یا اجرای همه آزمون‌ها عملی نباشد.

در درستی‌یابی صوری^۳، مدل سامانه و ویژگی مورد انتظار با زبان دقیق ریاضی بیان می‌شوند. سپس یک روش استدلالی یا الگوریتمی بررسی می‌کند آیا ویژگی برای رفتارهای مدل برقرار است یا خیر. واریسی مدل^۴ یکی از این روش‌هاست که فضای حالت مدل را به طور خودکار جست‌وجو می‌کند و در صورت نقض ویژگی معمولاً یک مثال نقض ارائه می‌دهد [۷، ۳]. تضمین حاصل نه درباره هر جزئیات جهان واقعی، بلکه درباره مدل، مشخصه و فرض‌های تعریف‌شده است؛ از این رو، آزمون و درستی‌یابی صوری مکمل یکدیگرند.

کاربرد روش‌های صوری به پژوهش دانشگاهی محدود نمانده است. مهندسان Amazon Web Services از سال ۲۰۱۱ مشخصه‌نویسی صوری و واریسی مدل با TLA+ را برای مسائل دشوار طراحی سامانه‌های توزیع‌شده بحرانی به کار گرفته‌اند [۱۶]. در Meta، تحلیل‌گر ایستای Infer با استدلال نمادین و منطق ریاضی، مسیرهای متعدد برنامه را بدون اجرای آن بررسی می‌کند و در زمان گزارش رسمی، هر ماه صدها خطای احتمالی پیش از انتشار رفع می‌شدند؛ البته این ابزار تضمین کامل درستی تابعی ارائه نمی‌کند [۱۳]. پروژه Everest در مایکروسافت نیز مؤلفه‌های رمزنگاری، تجزیه‌گرها و پروتکل‌های ارتباط امن را همراه با اثبات‌های ماشین‌بررسی‌شده برای

^۳Formal Verification

^۴Model Checking

استفاده عملی توسعه داده است [۳۰]. این نمونه‌ها طیفی از روش‌های صوری را نشان می‌دهند: از تحلیل خودکار برای یافتن رده‌ای از خطاها تا اثبات ویژگی‌های مشخص یک مؤلفه.

۴.۱ انفجار فضای حالت و جایگاه این پروژه

پوشش همه رفتارهای یک مدل هزینه دارد. اگر هر مؤلفه چند حالت داشته باشد، ترکیب هم‌زمان چند مؤلفه می‌تواند تعداد بسیار بیشتری حالت سراسری بسازد. صف پیام، مقدار متغیرها، ترتیب اجرای کنشگرها و مقدارهای زمانی نیز این فضا را گسترش می‌دهند. این پدیده انفجار فضای حالت^۵ نام دارد و یکی از موانع اصلی کاربرد واریسی مدل است [۷، ۳].

جایگاه این پروژه نسبت به این مسئله باید از همان ابتدا روشن باشد. هدف پروژه غلبه بر انفجار فضای حالت در هنگام تولید آن نیست. آنچه انفجار فضای حالت می‌سازد، یک سامانه انتقال زمان‌دار^۶ بسیار بزرگ است؛ و کاری که ابزار این پروژه انجام می‌دهد این است که همان سامانه بزرگ را، پس از آنکه ساخته شد، به سامانه‌ای کوچک‌تر تبدیل کند که برای پرسش مورد نظر کاربر رفتار یکسانی دارد. بنابراین حافظه و زمان لازم برای تولید فضای حالت تغییری نمی‌کند؛ آنچه تغییر می‌کند، اندازه چیزی است که پس از تولید باید تحلیل، مقایسه یا مشاهده شود.

این تمایز اهمیت عملی دارد. یک فضای حالت زمان‌دار با ده‌ها یا صدها حالت را به‌سختی می‌توان روی کاغذ دنبال کرد یا با مدل دیگری مقایسه کرد. اگر بتوان همان سامانه را به نسخه‌ای کوچک‌تر با رفتار قابل مشاهده یکسان تبدیل کرد، دنبال کردن چند ویژگی روی خود گراف و مقایسه دو مدل عملی می‌شود.

پایه این کاهش، یک مشاهده ساده است: همه جزئیات مدل برای هر پرسش تحلیلی اهمیت یکسان ندارند. فرض کنید هدف، بررسی توالی فرمان‌های قابل مشاهده یک کنترل‌کننده باشد. گام‌های داخلی محاسبه یا پیام‌هایی که در سطح انتزاع مورد نظر دیده نمی‌شوند، ممکن است مسیر اجرا را طولانی کنند بی‌آنکه مشاهده بیرونی تازه‌ای بسازند. اگر بتوان حالت‌هایی را که از دید رفتار مورد توجه تمایزپذیر نیستند شناسایی و ادغام کرد، تحلیل روی مدلی کوچک‌تر انجام می‌شود. شرط اساسی آن است که کاهش، همان جنبه رفتاری را که قرار است بررسی شود حفظ کند.

^۵State-Space Explosion

^۶Timed Transition System

در سامانه زمان‌دار، صرفاً حذف کنش داخلی کافی نیست. دو اجرا ممکن است از نظر توالی پیام‌های قابل مشاهده یکسان باشند، اما مدت متفاوتی داشته باشند. شبیه‌سازی دوسویه ضعیف زمان‌دار اجازه می‌دهد کنش‌های داخلی نادیده گرفته شوند، درحالی‌که رفتار قابل مشاهده و مقدار زمان سپری‌شده حفظ می‌شود [۵]، [۲۶]. رده‌های هم‌ارزی حاصل می‌توانند مبنای ساخت یک مدل خارج‌قسمتی کوچک‌تر باشند.

۵.۱ مقایسه پیاده‌سازی و مشخصه

یکی از کاربردهای اصلی واریسی مدل، و یکی از انگیزه‌های این پروژه، مقایسه دو مدل با یکدیگر است: یکی مدلی انتزاعی که نقش مشخصه را دارد و رفتار مطلوب را بیان می‌کند، و دیگری مدلی دقیق‌تر که نقش پیاده‌سازی را دارد. پرسش این است که آیا پیاده‌سازی، از دید ناظر بیرونی، همان رفتاری را نشان می‌دهد که مشخصه وعده داده است. پاسخ‌دادن به این پرسش نیازمند معیاری برای «رفتار یکسان» است، و هم‌ارزی‌های رفتاری دقیقاً همین معیار را فراهم می‌کنند.

در سامانه‌های زمان‌دار، مشخصه معمولاً جزئیات داخلی را نمی‌گوید و فقط کنش‌های قابل مشاهده و زمان‌بندی آن‌ها را مقید می‌کند، درحالی‌که پیاده‌سازی پر از گام‌های داخلی است. از این‌رو معیار مناسب برای این مقایسه، یک هم‌ارزی ضعیف است که گام‌های داخلی را نادیده بگیرد اما زمان را حفظ کند. همین معیار است که در این پروژه برای ادغام حالت‌ها هم به کار می‌رود؛ زیرا ادغام حالت‌های تمایزناپذیر و مقایسه دو مدل، دو روی یک سکه‌اند.

۶.۱ ایده الگوریتم

الگوریتمی که در این پروژه پیاده‌سازی شده است، بر ایده پالایش افراز^۷ استوار است: حالت‌ها ابتدا همه در یک رده قرار می‌گیرند و سپس هر رده‌ای که اعضایش رفتار متفاوتی نشان دهند شکسته می‌شود، تا جایی که هیچ رده‌ای دیگر قابل شکستن نباشد. این ایده از الگوریتم گروته و واندراخر برای محاسبه شبیه‌سازی دوسویه انشعابی^۸

^۷Partition Refinement

^۸Branching Bisimulation

روی سامانه‌های انتقال برچسب‌دار گرفته شده است [۱۰]؛ سهم این پروژه، تطبیق آن ایده با حالت زمان‌دار است، نه پیشنهاد یک هم‌ارزی یا الگوریتم تازه.

شبیه‌سازی دوسویه انشعابی، که فان‌گلایک و ویلاند آن را معرفی کرده‌اند، رابطه‌ای میان حالت‌های دو سامانه است که گام‌های داخلی را نادیده می‌گیرد اما —برخلاف تجربدهای ضعیف‌تر— اجازه نمی‌دهد آن گام‌های داخلی امکان‌های رفتاری را تغییر دهند: مسیر داخلی‌ای که در پاسخ به یک کنش پیموده می‌شود باید تماماً از حالت‌هایی بگذرد که با حالت مبدأ در همان رابطه‌اند [۳۲]. به این ترتیب، افزون بر توالی کنش‌های قابل مشاهده، محل انتخاب میان رفتارهای آینده نیز حفظ می‌شود. هم‌ارزی هدف این پروژه، چنان‌که در فصل ۲ تعریف می‌شود، شبیه‌سازی دوسویه ضعیف زمان‌دار است؛ آنچه از کار گروته و واندراخر گرفته شده، ساختار الگوریتمی پالایش افراز است. فصل ۳ این وام‌گیری و تفاوت‌های آن را دقیق‌تر شرح می‌دهد.

۷.۱ بستر ربکا و افرا

ربکا^۹ زبانی کنشگرمحور برای مدل‌سازی سامانه‌های هم‌روند و توزیع شده است. هر شیء واکنشی حالت و صف پیام خود را دارد و از طریق ارسال ناهمگام پیام با دیگر اشیا ارتباط برقرار می‌کند. ربکا زمان‌دار^{۱۰} سازوکارهایی برای تأخیر، مهلت و زمان محاسبه به این مدل می‌افزاید [۲۸، ۲۹، ۲۷]. این ساختار برای بیان شهودی یک سامانه واکنشی مناسب است، اما ترکیب صف‌ها، کنشگرها و زمان همان مسئله رشد فضای حالت را ایجاد می‌کند.

افرا^{۱۱} محیط مدل‌سازی و تحلیل خانواده ربکا است و با گردآوردن کامپایلر، واریسی گرامر مدل و ابزارهای نمایش، فضای حالت حاصل را برای تحلیل در اختیار کاربر می‌گذارد [۱۸، ۱۹]. ورودی پروژه حاضر کد منبع ربکا نیست، بلکه سامانه انتقال زمان‌داری است که این زنجیره با قالب statespace. تولید می‌کند. بنابراین مسئله در مرز میان تولید مدل معنایی و تحلیل آن قرار دارد: چگونه می‌توان با انتخاب تعامل‌های قابل مشاهده، مدل را بر اساس شبیه‌سازی دوسویه ضعیف زمان‌دار کاهش داد و هم‌زمان شهودی قابل واریسی برای حفظ رفتار ارائه کرد؟

^۹Rebeca

^{۱۰}Timed Rebeca

^{۱۱}Afra

۸.۱ هدف‌ها و پرسش‌های پروژه

هدف اصلی این پروژه تبدیل تعریف و شبه‌کد کاهش ضعیف زمان‌دار به ابزاری اجرایی برای فضای حالت افراست. این پایان‌نامه الگوریتم هم‌ارزی تازه‌ای پیشنهاد نمی‌کند؛ کار آن تطبیق تعریف موجود با قرارداد واقعی خروجی افرا، پیاده‌سازی فرایند کاهش و فراهم کردن شواهد قابل تکرار برای درستی خروجی است. پروژه سه پرسش را دنبال می‌کند:

۱. آیا می‌توان خروجی زمان‌دار افرا را بدون ویرایش دستی خواند و به یک نمایش مستقل از رابط کاربری افرا تبدیل کرد؟

۲. آیا می‌توان پس از پنهان‌سازی تعامل‌های انتخاب‌نشده، یک مدل خارج‌قسمتی کوچک‌تر ساخت و رابطه آن با ورودی را به‌طور مستقل واریسی کرد؟

۳. شکستن یال‌های تأخیر به گام‌های یک‌واحدی چه اثری بر نتیجه هم‌ارزی نمونه‌ها دارد و روش روی ورودی‌های موجود چه مقدار کاهش ایجاد می‌کند؟

برای پاسخ به این پرسش‌ها، هدف‌های اجرایی زیر تعیین شده‌اند:

- استخراج و مستندسازی بخش مورد نیاز از قرارداد فایل TTS افرا؛
- پیاده‌سازی پنهان‌سازی، پالایش تأخیر، ساخت گذار ضعیف و پالایش افرا؛
- ساخت و فشرده‌سازی سامانه انتقال خارج‌قسمتی^{۱۲} همراه با نگاشت حالت‌های ورودی؛
- توسعه واریسی‌گری مستقل برای آزمودن رابطه مدل تحویلی و ورودی؛
- ثبت مصنوعات، سنجه‌ها و آزمون‌های لازم برای تکرار ارزیابی.

¹²Quotient Transition System

۹.۱ دامنه و دستاوردهای پروژه

دامنه پیاده‌سازی به سامانه‌های انتقال متناهی با زمان گسسته و تأخیر صحیح نامنفی محدود است. ابزار خروجی TTS افرا را می‌خواند و از FTTS، زمان چگال، کامپایل مستقیم مدل ربکا و ادغام به‌صورت افزونه رابط کاربری افرا پشتیبانی نمی‌کند. در معنای پیش‌فرض، هر تأخیر چندواحدی به گام‌های یک‌واحدی شکسته می‌شود؛ بخش ۶.۳ نشان می‌دهد که این کار مستقیماً از ویژگی پیوستگی سامانه انتقال زمان‌دار می‌آید و بدون آن، تعریف مورد نظر روی گراف ذخیره‌شده قابل محاسبه نیست.

دست‌آورد پروژه یک ابزار مستقل جاوا با رابط خط فرمان، خواننده مقاوم در برابر ویژگی خاص فایل واقعی افرا، خط لوله کاهش قابل ردیابی و واریسی‌گر جداگانه است. نتیجه اصلی به‌صورت JSON بدون اتلاف، نگاشت افراز، نمایش سازگار با افرا، نمودار و سنج‌های اجرا ذخیره می‌شود [۱۵].

سهم اصلی این کار، وجود خود ابزار است. زنجیره ابزار ربکا و افرا فضای حالت زمان‌دار را تولید و نمایش می‌دهد، اما ابزاری که همان فضای حالت را نسبت به مجموعه‌ای از تعامل‌های قابل مشاهده و بر اساس شبیه‌سازی دوسویه ضعیف زمان‌دار کاهش دهد، پیش از این در دسترس کاربر این زنجیره نبوده است [۱۸، ۱۹، ۲۱]. ارزش این کار در پیوند زدن تعریف نظری با قرارداد واقعی داده افرا و نیازهای مهندسی این زنجیره است. ادعای مینی‌مال یا کانونی بودن مدل خروجی و نیز ادعای مقیاس‌پذیری صنعتی مطرح نمی‌شود.

۱۰.۱ روش کلی ارزیابی

پیاده‌سازی در سه سطح بررسی شده است. آزمون‌های واحد، اجزای خواندن فایل، معنای گذار ضعیف، پالایش افراز، ساخت خارج‌قسمت و واریسی رابطه را می‌سنجند. آزمون‌های انتهابه‌انتها برنامه بسته‌بندی شده را از بیرون اجرا و مصنوعات، کدهای خروج و قطعی بودن نتیجه را بررسی می‌کنند. در نهایت، ارزیابی روی دو دسته ورودی انجام شده است: چهار ورودی سنج‌های شامل یک نمونه کوچک و سه بیان از مطالعه موردی خانه هوشمند، و هفت فضای حالت که از مدل‌های واقعی ربکا و با همان زنجیره ابزار افرا تولید شده‌اند. دو نمونه خانه هوشمند، تأخیر ده‌واحدی را به ترتیب به $3 + \tau + 7$ و $7 + \tau + 3$ می‌شکنند تا حساسیت روش به شکستن یال‌های تأخیر آشکار شود. سنج‌های فصل ۴ مستقیماً از خروجی ثبت‌شده همین اجراها گزارش می‌شوند.

۱۱.۱ ساختار پایان‌نامه

فصل ۲ پیش‌نیازهای درستی‌یابی صوری، واریسی مدل، سامانه‌های واکنشی و زمان‌دار، ربکا و افرا، سامانه‌های انتقال و تعریف شبیه‌سازی دوسویه ضعیف زمان‌دار را ارائه می‌کند. فصل ۳ قرارداد ورودی، تصمیم‌های معنایی، مراحل الگوریتم، معماری پیاده‌سازی و روش اعتبارسنجی را شرح می‌دهد. فصل ۴ نتیجه کاهش، حساسیت معنایی، پوشش آزمون‌ها، زمان اجرا و تحلیل نتایج را گزارش می‌کند. در پایان، فصل ۵ به پرسش‌های پروژه پاسخ می‌دهد، دستاوردها و محدودیت‌ها را جمع‌بندی می‌کند و مسیرهای ادامه کار را پیشنهاد می‌دهد.

فصل ۲

مبانی نظری و تعاریف

این فصل مفاهیمی را معرفی می‌کند که برای دنبال کردن مسئله و روش این پروژه لازم‌اند. متن با فرض آشنایی خواننده با مباحث عمومی دوره کارشناسی مهندسی کامپیوتر نوشته شده است، نه با فرض آشنایی او با روش‌های صوری یا سیستم‌های زمان‌دار. از این رو، هر موضوع ابتدا به صورت شهودی و سپس، هر جا در ادامه به دقت ریاضی نیاز باشد، به صورت صوری بیان می‌شود. هدف ارائه یک مرجع جامع برای هریک از این حوزه‌ها نیست؛ خواننده‌ای که با یک موضوع آشناست می‌تواند از بخش مربوط به آن عبور کند.

برخی اصطلاحات این حوزه در منابع مختلف به تعریف‌های یکسانی اشاره نمی‌کنند. برای مثال، دامنه زمان ممکن است گسسته یا چگال باشد، کنش داخلی در یک منبع با τ و در منبع دیگر با ε نشان داده شود، و شبیه‌سازی دوسویه زمان‌دار می‌تواند قوی یا ضعیف باشد [۱، ۵، ۱۲، ۲۶]. این تفاوت‌ها صرفاً واژگانی نیستند و می‌توانند رفتار مدل و نتیجه تحلیل را تغییر دهند. بنابراین، تعریف‌های صوری این فصل قرارداد دقیق پایان‌نامه را تعیین می‌کنند؛ به ویژه، ساختار مرجع سراسر پایان‌نامه سامانه انتقال زمان‌دار است و تعریف ضعیف زمان‌دار، به همراه ویژگی‌های استاندارد رابطه گذار، از برار و همکاران گرفته شده است [۵].

مسیر فصل از چرایی درستی یابی نرم‌افزار آغاز می‌شود و پس از معرفی واری مدل و سیستم‌های بلادرنگ، مدل کنشگر، ربکا و افرا را معرفی می‌کند. سپس سامانه انتقال برچسب‌دار و سامانه انتقال زمان‌دار تعریف می‌شوند، هم‌ارزی‌های رفتاری—از جمله شبیه‌سازی دوسویه انشعابی که ایده الگوریتمی این پروژه از آن گرفته شده است—بررسی می‌شوند و تعریف شبیه‌سازی دوسویه ضعیف زمان‌دار ارائه می‌شود. در پایان نیز ارتباط این تعریف با افراز فضای حالت و کاهش مدل توضیح داده می‌شود.

۱.۲ از آزمون نرم‌افزار تا درستی‌یابی صوری

نرم‌افزار معمولاً با مجموعه‌ای از آزمون‌های واحد، یکپارچه و سیستمی ارزیابی می‌شود. هر آزمون، اجرای سامانه را برای یک ورودی و شرایط مشخص مشاهده می‌کند و می‌تواند وجود یک خطا را آشکار سازد. با این حال، موفقیت مجموعه‌ای متناهی از آزمون‌ها به‌تنهایی نشان نمی‌دهد که سامانه در همه ورودی‌ها و همه ترتیب‌های اجرای ممکن درست رفتار می‌کند. این محدودیت به‌ویژه در سامانه‌های هم‌روند و واکنشی مهم است؛ زیرا زمان‌بندی متفاوت رویدادها می‌تواند تعداد بسیار زیادی رفتار ممکن پدید آورد [۳].

روش‌های صوری^۱ مجموعه‌ای از زبان‌ها، مدل‌ها و فنون مبتنی بر ریاضیات‌اند که برای توصیف دقیق، تحلیل و ساخت سامانه‌های نرم‌افزاری و سخت‌افزاری به کار می‌روند. استفاده از این روش‌ها ابهام زبان طبیعی را کاهش می‌دهد و امکان استدلال نظام‌مند درباره رفتار سامانه را فراهم می‌کند. در این پایان‌نامه، جریان به‌کارگیری روش‌های صوری در سه بخش مکمل در نظر گرفته می‌شود [۷، ۳، ۲۸]:

۱. مشخصه‌نویسی صوری^۲: نیازمندی‌ها، ساختار یا رفتار مورد انتظار سامانه با یک زبان دقیق ریاضی بیان می‌شود؛

۲. درستی‌یابی صوری: بررسی می‌شود که مدل یا پیاده‌سازی، مشخصه صوری تعیین‌شده را برآورده می‌کند یا نه؛

۳. پالایش صوری^۳ یا توسعه صوری^۴: مشخصه انتزاعی طی گام‌های کنترل‌شده به طراحی یا پیاده‌سازی دقیق‌تر تبدیل می‌شود، به‌گونه‌ای که ویژگی‌های لازم در هر گام حفظ شوند.

این سه بخش الزاماً خطی و مستقل از یکدیگر نیستند؛ نتیجه درستی‌یابی ممکن است به اصلاح مشخصه بینجامد و هر گام پالایش نیز تعهدهای درستی‌یابی تازه‌ای ایجاد کند.

درستی‌یابی نرم‌افزار^۵ مفهومی گسترده‌تر از اجرای آزمون‌هاست و بررسی می‌کند که یک مصنوع نرم‌افزاری نیازمندی‌های تعیین‌شده را برآورده می‌کند یا نه. در درستی‌یابی صوری، خود سامانه و ویژگی مورد انتظار با زبان ریاضی توصیف می‌شوند و درستی رابطه میان آن‌ها با استدلال ریاضی یا یک روش الگوریتمی بررسی می‌شود.

¹Formal Method

²Formal Specification

³Formal Refinement

⁴Formal Development

⁵Software Verification

بنابراین، یک مسئله درستی‌یابی صوری معمولاً سه جزء دارد: مدلی از سامانه، مشخصه‌ای صوری از رفتار مطلوب یا نامطلوب، و روشی برای اثبات یا رد برقراری آن مشخصه [۷، ۲۹].

نتیجه درستی‌یابی همواره نسبت به مدل و مشخصه‌ای است که تحلیل شده‌اند. اثبات درست‌بودن یک مدل، خودبه‌خود خطای پیاده‌سازی، فرض نادرست محیط، یا نیازمندی‌ای را که در مشخصه نیامده است حذف نمی‌کند. به همین دلیل، آزمون و درستی‌یابی صوری جانشین کامل یکدیگر نیستند: آزمون رفتار پیاده‌سازی واقعی را در اجراهای انتخاب‌شده می‌سنجد، درحالی‌که درستی‌یابی صوری درباره همه رفتارهای مدل، در محدوده فرض‌های صریح آن، استدلال می‌کند [۳].

دو دسته رایج از ویژگی‌ها عبارت‌اند از ویژگی‌های ایمنی و زنده‌بودن. ویژگی ایمنی بیان می‌کند که «رخداد بدی اتفاق نمی‌افتد»؛ برای نمونه، دو فرایند هم‌زمان وارد یک بخش بحرانی نمی‌شوند. ویژگی زنده‌بودن بیان می‌کند که «رخداد مطلوبی سرانجام اتفاق می‌افتد»؛ برای نمونه، هر درخواست پذیرفته‌شده سرانجام پاسخی دریافت می‌کند. بیان دقیق این ویژگی‌ها ممکن است با منطق‌های زمانی انجام شود، اما جزئیات آن‌ها برای تعریف‌های این فصل لازم نیست [۳].

۲.۲ واریسی مدل

واریسی مدل یکی از روش‌های خودکار درستی‌یابی صوری برای سامانه‌هایی است که رفتارشان به صورت حالت و گذار مدل شده است. اگر M مدل سامانه و φ ویژگی مورد نظر باشد، پرسش اصلی به شکل

$$M \models \varphi$$

نوشته می‌شود: آیا همه رفتارهای مدل M مشخصه φ را ارضا می‌کنند؟ الگوریتم واریسی مدل فضای حالت قابل‌دسترسی را به طور نظام‌مند بررسی می‌کند. اگر ویژگی برقرار نباشد، ابزار معمولاً یک مثال نقض ارائه می‌کند؛ یعنی اجرایی از مدل که چگونگی نقض ویژگی را نشان می‌دهد [۷، ۳].

مزیت واریسی مدل، خودکاربودن و پوشش همه رفتارهای مدلی است که در محدوده تحلیل قرار دارد. مانع اصلی آن انفجار فضای حالت است: تعداد حالت‌های مدل ممکن است با افزایش مؤلفه‌ها، متغیرها یا ترتیب‌های ممکن اجرا به سرعت رشد کند. در چنین شرایطی، حتی اگر هر گام بررسی ساده باشد، نگهداری و پیمایش همه

حالت‌ها پرهزینه یا ناممکن می‌شود. کاهش مدلی که رفتارهای مرتبط با ویژگی مورد بررسی را حفظ کند، یکی از راه‌های مقابله با این مسئله است [۲۸، ۷].

۳.۲ سیستم‌های واکنشی و بلادرنگ

یک سیستم واکنشی در تعامل پیوسته با محیط قرار دارد: ورودی دریافت می‌کند، حالت داخلی خود را تغییر می‌دهد و در پاسخ، خروجی یا رویدادی تولید می‌کند. پروتکل‌های ارتباطی، کنترل‌کننده‌ها و سامانه‌های توزیع‌شده نمونه‌هایی از این الگو هستند. در یک سامانه هم‌روند، چند مؤلفه می‌توانند به صورت مستقل پیشرفت کنند و ترتیب نسبی کنش‌های آن‌ها بخشی از رفتار سامانه است [۲۸، ۲۹].

در سیستم بلادرنگ، درست بودن پاسخ فقط به مقدار منطقی آن وابسته نیست و زمان تولید پاسخ نیز اهمیت دارد. برای نمونه، فرمان درست یک کنترل‌کننده اگر پس از مهلت لازم صادر شود می‌تواند بی‌فایده باشد. آلور و دیل برای مدل کردن چنین محدودیت‌هایی، رویدادهای یک رد را با زمان حقیقی وقوع آن‌ها همراه کردند و صورت اولیه اتوماتای زمان‌دار را ارائه دادند [۱].

برای تحلیل چنین سامانه‌ای باید گذر زمان نیز در مدل قابل مشاهده باشد. یک مدل صرفاً ترتیبی ممکن است نشان دهد که کنش a پیش از b رخ داده است، اما بیان نمی‌کند فاصله زمانی میان آن دو چقدر بوده یا آیا یک مهلت رعایت شده است. مدل معنایی‌ای که در بخش ۶.۲ تعریف می‌شود همین کمبود را جبران می‌کند: در آن، گذشت زمان خود یک نوع گذار است و در کنار کنش‌ها ثبت می‌شود [۵].

۴.۲ مدل کنشگر، ربکا و افرا

مدل کنشگر^۶ سامانه هم‌روند را به مجموعه‌ای از موجودیت‌های مستقل تقسیم می‌کند. هر کنشگر حالت داخلی خود را در اختیار دارد و با دریافت و ارسال ناهمگام پیام با کنشگرهای دیگر تعامل می‌کند. این جداسازی، واحد هم‌روندی را خود کنشگر قرار می‌دهد و برای مدل کردن سامانه‌های توزیع‌شده و واکنشی مناسب است [۲۸، ۲۹].

^۶ Actor Model

نام ربکا کوتاه‌شده Reactive Objects Language است. ربکا زبانی کنشگرمحور برای مدل‌سازی و درستی‌یابی سامانه‌های هم‌روند و توزیع‌شده است. این زبان با هدف نزدیک کردن مدل‌سازی صوری به ساختارهای آشناتر مهندسی نرم‌افزار طراحی شده و نحو آن به جاوا شباهت دارد. در ربکا، یک reactive class الگوی ساخت اشیای واکنشی است و هر نمونه آن یک rebec نامیده می‌شود. هر ربکا حالت محصور، رشته اجرای مستقل و صف پیام خود را دارد. دریافت یک پیام اجرای message server متناظر را فعال می‌کند و ارتباط میان ربکاها از طریق ارسال ناهمگام و نامسدودکننده پیام انجام می‌شود [۲۸، ۲۹].

تعریف صوری نحو و معنای ربکا و روش‌های درستی‌یابی ترکیبی آن در رساله دکتری مرجان سیرجانی ارائه شده است. در آن رساله، ساختار محصور و ارتباط ناهمگام ربکاها مبنایی برای تجزیه مدل، تجرید و کاهش فضای حالت قرار می‌گیرد. مقاله سیرجانی و همکاران نیز این چارچوب را با سه جزء مدل رفتاری، زبان مشخصه و روش تحلیل توضیح می‌دهد و درستی تجرید پیشنهادی خود را با یک رابطه شبیه‌سازی ضعیف استدلال می‌کند [۲۸، ۲۹].

ربکای زمان‌دار ربکا را با سازوکارهای بیان تأخیرهای محاسباتی و شبکه‌ای و مهلت زمانی گسترش می‌دهد تا رفتار سامانه‌های توزیع‌شده ناهمگام زمان‌دار مدل شود. معنای این زبان به صورت صوری تعریف شده است و مدل حاصل می‌تواند برای تحلیل رفتار زمانی به کار رود [۲۷].

افرا محیط یکپارچه مدل‌سازی و درستی‌یابی خانواده ربکا است. افرا امکانات ویرایش مدل و ویژگی، اجرای واریسی مدل، نمایش نتیجه و مشاهده مثال نقض را یکجا فراهم می‌کند و اجزایی مانند کامپایلر ربکا و Rebeca Model Checker را در یک جریان کاری گرد هم می‌آورد [۱۸]. کد منبع اجزای اصلی این زنجیره، از جمله افرا، کامپایلر و واریسی‌گر مدل، در مخازن رسمی گروه پژوهشی ربکا منتشر شده است [۲۳، ۱۹، ۲۰، ۲۱].

در این پروژه، اهمیت افرا فقط به رابط کاربری آن محدود نیست: مدل معنایی تولیدشده برای یک مدل ربکای زمان‌دار، فضای حالتی را فراهم می‌کند که کاهش رفتاری روی آن تعریف می‌شود. بنابراین، ربکا زبان مدل‌سازی، افرا و واریسی‌گر مدل ابزار ساخت و تحلیل مدل معنایی، و شبیه‌سازی دوسویه ضعیف زمان‌دار معیار تشخیص حالت‌های قابل ادغام‌اند [۱۸، ۲۶].

۵.۲ سامانه انتقال برچسب‌دار

برای تحلیل الگوریتمی یک مدل باید رفتار آن به صورت یک ساختار ریاضی مشخص بیان شود. ساده‌ترین چنین ساختاری سامانه انتقال برچسب‌دار^۷ است: مجموعه‌ای از حالت‌ها، یک حالت آغازین و مجموعه‌ای از گذارهای برچسب‌دار میان حالت‌ها. این ساختار همان چیزی است که یک واریسی گر مدل می‌سازد و پیمایش می‌کند [۳، ۱۴].

تعریف ۱.۵.۲ (سامانه انتقال برچسب‌دار). یک سامانه انتقال برچسب‌دار چندتایی

$$S = (Q, q_0, \Lambda, \rightarrow)$$

است که در آن Q مجموعه حالت‌ها، $q_0 \in Q$ حالت آغازین، Λ مجموعه متناهی برچسب‌ها و

$$\rightarrow \subseteq Q \times \Lambda \times Q$$

رابطه گذار است. به جای $(q, \mu, q') \in \rightarrow$ می‌نویسیم $q \xrightarrow{\mu} q'$ و می‌گوییم سامانه می‌تواند با انجام گذاری با برچسب μ از حالت q به حالت q' برود [۳، ۱۴].

در این پایان‌نامه، مجموعه برچسب‌ها به دو بخش تقسیم می‌شود: مجموعه Σ از کنش‌های قابل مشاهده که ناظر بیرونی آن‌ها را می‌بیند، و یک برچسب ویژه به نام کنش داخلی^۸ که با τ نشان داده می‌شود و بیانگر گامی است که از بیرون دیده نمی‌شود. مجموعه $\Sigma_\tau = \Sigma \cup \{\tau\}$ را برای اختصار به کار می‌بریم. نماد ε در سراسر این پایان‌نامه منحصرراً برای کلمه تهی استفاده می‌شود و هرگز به معنای کنش داخلی نیست؛ برخی منابع، از جمله برار و همکاران، کنش داخلی را با ε نشان می‌دهند [۵، ۱۴].

یک اجرا از حالت q دنباله‌ای متناوب از حالت‌ها و گذارهاست:

$$\rho = q_0 \xrightarrow{\mu_1} q_1 \xrightarrow{\mu_2} q_2 \dots$$

^۷Labeled Transition System

^۸Internal Action

با کنارگذاشتن نام حالت ها، دنباله برچسب های $\mu_1 \mu_2 \dots$ به دست می آید و اگر تنها کنش های قابل مشاهده نگه داشته شوند، این دنباله رد^۹ نامیده می شود. تفاوت میان اجرا و رد مهم است: چند اجرای متفاوت می توانند رد یکسانی تولید کنند، درحالی که محل انتخاب میان رفتارهای آینده در آن ها یکسان نباشد [۳۲، ۳].

۶.۲ سامانه انتقال زمان دار

سامانه انتقال برچسب دار فقط ترتیب کنش ها را ثبت می کند. برای مدل کردن یک سامانه بلادرنگ باید گذر زمان نیز در خود ساختار حضور داشته باشد. سامانه انتقال زمان دار دقیقاً همین کار را می کند: افزون بر گذارهای کنشی، دسته ای از گذارها صرفاً گذشت یک مدت زمان را نشان می دهند. این ساختار، و نه اتوماتای زمان دار، مدل مرجع سراسر این پایان نامه است؛ زیرا ورودی ابزار ساخته شده در این پروژه نیز دقیقاً یک سامانه انتقال زمان دار متناهی است که زنجیره ابزار ربکا و افرا تولید می کند.

تعریف زیر از برار و همکاران گرفته شده است [۵]. تنها تفاوت، نمادگذاری کنش داخلی است که مطابق قرارداد این فصل با τ نوشته می شود، و دامنه زمان که در اینجا به صورت یک پارامتر $\mathbb{T}$ باقی می ماند تا هم زمان چگال $\mathbb{R}_{\geq 0}$ و هم زمان گسسته $\mathbb{N}$ را در بر بگیرد.

تعریف ۱.۶.۲ (سامانه انتقال زمان دار). یک سامانه انتقال زمان دار روی دامنه زمان $\mathbb{T}$ و الفبای کنش های قابل مشاهده Σ ، چندتایی

$$S = (Q, q_0, \rightarrow)$$

است که Q مجموعه پیکربندی ها، $q_0 \in Q$ پیکربندی آغازین و $\rightarrow$ رابطه گذار است. هر گذار یکی از دو نوع زیر است:

- حرکت تأخیری $q \xrightarrow{d} q'$ با $d \in \mathbb{T}$ ، که گذشت دقیقاً d واحد زمان را نشان می دهد؛
- حرکت گسسته $q \xrightarrow{a} q'$ با $a \in \Sigma_\tau$ ، که اجرای یک کنش قابل مشاهده یا یک کنش داخلی را نشان می دهد.

[۵]

^۹Trace

حرکت گسسته آنی است و زمانی مصرف نمی‌کند؛ کل زمان سپری‌شده در یک اجرا فقط از حرکت‌های تأخیری می‌آید. افزون بر این، رابطه $\rightarrow$ باید چهار ویژگی استاندارد زیر را داشته باشد. این ویژگی‌ها در ادامه پایان‌نامه صریحاً استفاده می‌شوند، به‌ویژه در بخش ۶.۳ که توضیح می‌دهد چرا یال‌های تأخیر به گام‌های یک‌واحدی شکسته می‌شوند.

تعریف ۲.۶.۲ (ویژگی‌های استاندارد رابطه گذار زمان‌دار). برای هر سامانه انتقال زمان‌دار $S = (Q, q_0, \rightarrow)$ چهار ویژگی زیر برقرارند [۵]:

قطعیت زمانی: اگر $q \xrightarrow{d} q'$ و $q \xrightarrow{d} q''$ با $d \in \mathbb{T}$ ، آنگاه $q' = q''$ ؛ یعنی گذشت یک مدت معین از یک پیکربندی، مقصد یکتایی دارد.

تأخیر صفر: برای هر $q \in Q$ رابطه $q \xrightarrow{0} q$ برقرار است؛ یعنی گذشت زمان صفر هیچ تغییری ایجاد نمی‌کند.

جمع‌پذیری: اگر $q \xrightarrow{d} q'$ و $q' \xrightarrow{d'} q''$ با $d, d' \in \mathbb{T}$ ، آنگاه $q \xrightarrow{d+d'} q''$ ؛ یعنی دو انتظار پیاپی با یک انتظار به طول مجموع آن‌ها یکی است.

پیوستگی: اگر $q \xrightarrow{d} q'$ و $d = d' + d''$ با $d', d'' \in \mathbb{T}$ ، آنگاه پیکربندی q'' وجود دارد که

$$q \xrightarrow{d'} q'' \xrightarrow{d''} q';$$

یعنی یک انتظار به طول d را می‌توان در هر لحظه میانی مشاهده کرد و به دو انتظار کوتاه‌تر شکست.

دو ویژگی آخر یکدیگر را کامل می‌کنند: جمع‌پذیری اجازه می‌دهد چند انتظار پیاپی در هم ادغام شوند و پیوستگی اجازه می‌دهد یک انتظار بلند به قطعه‌های کوتاه‌تر شکسته شود. برار و همکاران یادآوری می‌کنند که این ویژگی‌ها نخستین بار در چارچوب جبر فرایندهای زمان‌دار به‌صورت صوری معرفی شده‌اند و در سامانه‌های انتقال زمان‌دار مورد بررسی آنان نیز برقرارند [۵].

باید توجه داشت که این ویژگی‌ها درباره خود سامانه انتقال زمان‌دار برقرارند، نه لزوماً درباره گراف متناهی ذخیره‌شده‌ای که آن را نمایش می‌دهد. خروجی فضای حالت افرا یک گراف متناهی است که فقط تعدادی یال تأخیر با مدت‌های معین را صریحاً ثبت می‌کند؛ برای نمونه، یک یال با مدت ۵ ثبت می‌شود اما لحظه‌های میانی

آن به صورت حالت جداگانه در فایل حضور ندارند. بخش ۶.۳ نشان می دهد که این فاصله میان نمایش ذخیره شده و سامانه انتقال زمان داری که آن را توصیف می کند، دقیقاً منشأ یکی از تصمیم های اصلی پیاده سازی است. با داشتن ویژگی های بالا، یک اجرا از S دنباله ای متناهی یا نامتناهی از حرکات هاست که در آن کنش های گسسته و مدت ها یکی در میان می آیند:

$$\rho = q_0 \xrightarrow{d_0} q'_0 \xrightarrow{a_0} q_1 \xrightarrow{d_1} q'_1 \xrightarrow{a_1} \dots$$

برای یک اجرای متناهی از q به q' می نویسیم $q \xrightarrow{w} q'$ که در آن $w = d_0 a_0 d_1 a_1 \dots$ دنباله برچسب های اجراست.

تعریف ۳.۶.۲ (کلمه بی زمان و مدت یک اجرا). برای اجرای ρ با دنباله برچسب های w :

- $\text{Untimed}(\rho)$ کلمه ای است که از چسباندن برچسب های کنشی $a_0 a_1 \dots$ به دست می آید، به طوری که کنش های داخلی حذف می شوند؛ پس این کلمه فقط از کنش های قابل مشاهده Σ ساخته شده است؛
- $\text{Duration}(\rho) = \sum_{i=0}^{|\rho|} d_i$ مجموع همه مدت های اجراست، که در آن $|\rho|$ طول اجراست.

[۵]

۷.۲ هم ارزی های رفتاری روی سامانه انتقال برچسب دار

پیش از پرداختن به حالت زمان دار، لازم است روشن شود «رفتار یکسان» در حالت بدون زمان چه معنایی دارد؛ زیرا الگوریتم این پروژه، چنان که در بخش ۸.۳ توضیح داده می شود، بر ایده یکی از همین هم ارزی ها بنا شده است. برابری ردها تنها مجموعه دنباله های قابل مشاهده را مقایسه می کند و می تواند محل انتخاب میان رفتارهای آینده را نادیده بگیرد. شبیه سازی رابطه ای جهت دار است: هر گام یک سامانه باید در سامانه دیگر قابل تقلید باشد. شبیه سازی دوسویه^{۱۰} این شرط را در هر دو جهت برقرار می کند و در نتیجه ساختار انشعاب رفتار را نیز حفظ می کند [۳، ۱۴].

¹⁰Bisimulation

تعریف ۱.۷.۲ (شبیه‌سازی دوسویه قوی). فرض کنید S_1 و S_2 دو سامانه انتقال برچسب‌دار با مجموعه برچسب مشترک Λ باشند. رابطه $R \subseteq Q_1 \times Q_2$ یک شبیه‌سازی دوسویه قوی است، هرگاه برای هر $(p, q) \in R$ و هر $\mu \in \Lambda$:

۱. اگر $p \xrightarrow{\mu}_1 p'$ ، حالتی مانند q' وجود داشته باشد که $q \xrightarrow{\mu}_2 q'$ و $(p', q') \in R$ ؛

۲. اگر $q \xrightarrow{\mu}_2 q'$ ، حالتی مانند p' وجود داشته باشد که $p \xrightarrow{\mu}_1 p'$ و $(p', q') \in R$.

[۳، ۱۴]

در تعریف قوی، کنش داخلی نیز مانند هر کنش دیگری باید جداگانه پاسخ داده شود. هنگام کاهش مدل، این حساسیت جزئیاتی را حفظ می‌کند که از دید ناظر بیرونی اهمیتی ندارند. دو راه شناخته‌شده برای تجرید از کنش داخلی وجود دارد. شبیه‌سازی دوسویه ضعیف^{۱۱}، که میلتر معرفی کرده است، اجازه می‌دهد یک کنش قابل مشاهده با دنباله‌ای از کنش‌های داخلی پیش و پس از آن پاسخ داده شود و کنش داخلی با دنباله دلخواهی از کنش‌های داخلی [۱۴]. شبیه‌سازی دوسویه انشعابی، که فان‌گلابیک و ویلاندر معرفی کرده‌اند، شرط سخت‌گیرانه‌تری می‌گذارد: در طول کنش‌های داخلی پاسخ، رده رفتاری نباید تغییر کند و بنابراین ساختار انشعاب نیز حفظ می‌شود [۳۲].

تعریف ۲.۷.۲ (شبیه‌سازی دوسویه انشعابی). فرض کنید S_1 و S_2 دو سامانه انتقال برچسب‌دار با برچسب‌های Σ_τ باشند و $\Rightarrow$ بستار بازتابی-تعدی رابطه $\xrightarrow{\tau}$ باشد. رابطه متقارن $R \subseteq Q_1 \times Q_2$ یک شبیه‌سازی دوسویه انشعابی است، هرگاه برای هر $(p, q) \in R$ و هر گذار $p \xrightarrow{\mu} p'$ با $\mu \in \Sigma_\tau$ یکی از دو حالت زیر برقرار باشد:

۱. $\mu = \tau$ و $(p', q) \in R$ ؛ یعنی گام داخلی p اصلاً پاسخی لازم ندارد، زیرا رده رفتاری را تغییر نداده است؛

۲. حالت‌های q_1 و q' وجود داشته باشند که

$$q \Rightarrow q_1 \xrightarrow{\mu} q', \quad (p, q_1) \in R, \quad (p', q') \in R.$$

شرط کلیدی، عضویت $(p, q_1) \in R$ است: همه حالت‌های میانی مسیر داخلی باید با خود p در رابطه بمانند، یعنی هیچ‌یک از آن گام‌های داخلی نباید امکان‌های رفتاری را تغییر داده باشد [۳۲].

¹¹Weak Bisimulation

تفاوت این دو تجرید در همان شرط $(p, q_1) \in R$ است. در شبیه‌سازی دوسویه ضعیف، مسیر داخلی پاسخ می‌تواند از حالت‌هایی بگذرد که رفتار متفاوتی دارند؛ در شبیه‌سازی دوسویه انشعابی چنین چیزی مجاز نیست. به همین دلیل، شبیه‌سازی دوسویه انشعابی هم‌ارزی ظریف‌تری است و ویژگی‌های بیشتری را حفظ می‌کند [۳۲]. اهمیت این هم‌ارزی برای پروژه حاضر جنبه الگوریتمی دارد. گروته و واندراخر برای محاسبه شبیه‌سازی دوسویه انشعابی الگوریتمی مبتنی بر پالایش افراز ارائه کردند که از افرازی درشت آغاز می‌کند و بلوک‌ها را تا رسیدن به افرازی پایدار می‌شکند؛ این الگوریتم پیچیدگی زمانی $O(n(n+m))$ دارد که n تعداد حالت‌ها و m تعداد گذارهاست [۱۰]. ایده پایه این خانواده از الگوریتم‌ها به کار پیچ و تارجان درباره مسئله درشت‌ترین افراز رابطه‌ای بازمی‌گردد [۱۷]، و نسخه‌های بعدی پیچیدگی را به $O(m \log n)$ بهبود داده‌اند [۹]. الگوریتمی که در فصل ۳ برای کاهش زمان‌دار ساخته می‌شود، بر همین ایده پالایش افراز بنا شده است؛ بخش ۸.۳ پیش از ارائه الگوریتم پروژه، این ایده را دقیق‌تر شرح می‌دهد.

۸.۲ شبیه‌سازی دوسویه زمان‌دار

در یک سامانه انتقال زمان‌دار، برچسب یک گذار می‌تواند یک مدت نیز باشد. بنابراین هم‌ارزی رفتاری باید افزون بر کنش‌ها، گذر زمان را نیز حفظ کند. ساده‌ترین صورت این کار، تعریف قوی است که هر گذار را جداگانه و با همان برچسب می‌سنجد.

تعریف ۱.۸.۲ (شبیه‌سازی دوسویه زمان‌دار قوی). فرض کنید $S_1 = (Q_1, q_{01}, \rightarrow_1)$ و $S_2 = (Q_2, q_{02}, \rightarrow_2)$ (دو سامانه انتقال زمان‌دار روی دامنه زمان $\mathbb{T}$ و الفبای Σ باشند. رابطه $R \subseteq Q_1 \times Q_2$ یک شبیه‌سازی دوسویه زمان‌دار قوی است، هرگاه برای هر $(p, q) \in R$ و هر $\mu \in \Sigma \cup \mathbb{T}$ شروط زیر برقرار باشند:

$$۱. \text{ اگر } p \xrightarrow{\mu}_1 p', \text{ حالتی مانند } q' \text{ وجود داشته باشد که } q \xrightarrow{\mu}_2 q' \text{ و } (p', q') \in R;$$

$$۲. \text{ اگر } q \xrightarrow{\mu}_2 q', \text{ حالتی مانند } p' \text{ وجود داشته باشد که } p \xrightarrow{\mu}_1 p' \text{ و } (p', q') \in R.$$

اگر افزون بر این $(q_{01}, q_{02}) \in R$ باشد، دو سامانه شبیه‌سازی دوسویه زمان‌دار قوی اند [۶، ۱۲].

تعریف قوی برای مقایسه دقیق دو مدل مفید است، اما برای کاهش مدل مناسب نیست: هر گام داخلی و هر تکه‌شدن یک انتظار، تمایزی می‌سازد که ناظر بیرونی آن را نمی‌بیند. تعریف ضعیف همین تمایزها را کنار

می‌گذارد. برای بیان آن، ابتدا رابطه گذار ضعیف تعریف می‌شود.

تعریف ۲.۸.۲ (رابطه گذار ضعیف). برای سامانه انتقال زمان‌دار $S = (Q, q_0, \rightarrow)$ ، رابطه

$$\Rightarrow \subseteq Q \times (\Sigma \cup \mathbb{T}) \times Q$$

برای $a \in \Sigma$ و $d \in \mathbb{T}$ چنین تعریف می‌شود:

$$\bullet \quad q \xRightarrow{d} q' \text{ اگر و تنها اگر اجرایی مانند } \rho = q \xrightarrow{w} q' \text{ وجود داشته باشد با}$$

$$\text{Untimed}(\rho) = \varepsilon, \quad \text{Duration}(\rho) = d;$$

$$\bullet \quad q \xRightarrow{a} q' \text{ اگر و تنها اگر اجرایی مانند } \rho = q \xrightarrow{w} q' \text{ وجود داشته باشد با}$$

$$\text{Untimed}(\rho) = a, \quad \text{Duration}(\rho) = \circ.$$

[۵]

به بیان دیگر، یک گام ضعیف زمانی هر اجرایی است که هیچ کنش قابل مشاهده‌ای در آن رخ ندهد و مجموع مدت‌هایش دقیقاً d باشد؛ این اجرا می‌تواند هر تعداد گام داخلی و هر تعداد حرکت تأخیری داشته باشد. یک گام ضعیف کنشی نیز اجرایی است که دقیقاً یک کنش قابل مشاهده دارد و زمانی در آن نمی‌گذرد، اما می‌تواند گام‌های داخلی پیش و پس از آن کنش را در خود جمع کند.

تعریف ۳.۸.۲ (شبیه‌سازی دوسویه ضعیف زمان‌دار). فرض کنید $S_1 = (Q_1, q_{01}, \rightarrow_1)$ و $S_2 = (Q_2, q_{02}, \rightarrow_2)$

(دو سامانه انتقال زمان‌دار باشند و $\approx$ رابطه‌ای دوتایی روی $Q_1 \times Q_2$ باشد. رابطه $\approx$ یک شبیه‌سازی دوسویه

ضعیف زمان‌دار میان S_1 و S_2 است، هرگاه $q_{01} \approx q_{02}$ و برای هر $\mu \in \Sigma \cup \mathbb{T}$:

$$۱. \text{ اگر } q_1 \approx q_2 \text{ و } q_1 \xRightarrow{\mu}_1 q'_1 \text{، آنگاه } q'_1 \text{ وجود داشته باشد که } q'_1 \xRightarrow{\mu}_2 q'_2 \text{ و } q_2 \approx q'_2؛$$

$$۲. \text{ به عکس، اگر } q'_2 \xRightarrow{\mu}_2 q_2 \text{ و } q_2 \approx q_1 \text{، آنگاه } q'_1 \text{ وجود داشته باشد که } q'_1 \xRightarrow{\mu}_1 q_1 \text{ و } q_1 \approx q'_1.$$

دو سامانه S_1 و S_2 شبیه سازی دوسویه ضعیف زمان دار دارند و می نویسیم $S_1 \approx_W S_2$ ، هرگاه چنین رابطه ای میان آن ها وجود داشته باشد [۵، ۲۶].

آنچه در تعریف ضعیف باید یکسان بماند، کنش قابل مشاهده، مجموع زمان سپری شده و امکان ادامه رفتار از حالت های مقصد است؛ نه تعداد و محل گام های داخلی. همین ویژگی است که اجازه می دهد حالت هایی که فقط در جزئیات داخلی تفاوت دارند در یک رده قرار گیرند، بی آنکه تمایز زمانی قابل مشاهده از میان برود [۵، ۲۶].

۹.۲ زمان گسسته و زمان چگال

دامنه زمان $\mathbb{T}$ در تعریف ۱.۶.۲ می تواند چگال، مانند $\mathbb{R}_{\geq 0}$ ، یا گسسته، مانند $\mathbb{N}$ ، باشد. انتخاب دامنه بخشی از مدل سازی است. در زمان چگال، حتی یک مدل نحوی کوچک می تواند بی نهایت پیکربندی داشته باشد؛ از این رو تحلیل الگوریتمی به بازنمایی های نمادین مانند گراف ناحیه یا گراف zone و ساختار داده ماتریس کران تفاضل نیاز دارد [۲، ۱۱، ۴، ۱۲].

مدل معنایی ای که زنجیره ربکا و افرا برای یک مدل ربکای زمان دار تولید می کند، زمان را گسسته و مدت ها را عدد صحیح نامنفی در نظر می گیرد [۲۷]. بنابراین در سراسر این پایان نامه $\mathbb{T} = \mathbb{N}$ گرفته می شود و همه مدت ها اعداد صحیح نامنفی اند. این انتخاب یک سادگی مهم به همراه دارد که فصل ۳ از آن استفاده می کند: کوچک ترین واحد زمان وجود دارد و هر تأخیر را می توان به تعداد متناهی گام یک واحدی تجزیه کرد. بازنمایی نمادین زمان چگال در این پروژه پیاده سازی نشده است و در فصل ۵ به عنوان مسیر ادامه کار مطرح می شود.

۱۰.۲ هم ارزی، افراز و کاهش مدل

شبیه سازی دوسویه یک رابطه هم ارزی است؛ یعنی حالت ها را می توان بر اساس آن به رده های جدا از هم تقسیم کرد. چنین تقسیم بندی ای افراز^{۱۲} نام دارد. در هر بلوک افراز، حالت هایی قرار می گیرند که مطابق تعریف انتخاب شده از دید رفتاری تمایز پذیر نیستند [۳، ۳۲].

¹²Partition

با جایگزین کردن هر بلوک با یک حالت و انتقال گذارها میان بلوک‌ها، مدل خارج‌قسمتی به دست می‌آید. اگر افراز نسبت به شبیه‌سازی دوسویه ضعیف زمان‌دار پایدار باشد، ادغام حالت‌های یک بلوک کنش‌های قابل مشاهده، زمان کل و ساختار انشعاب مورد نظر تعریف را حفظ می‌کند. هدف کاهش مدل، ساختن چنین نمایش کوچک‌تری است؛ نه حذف دلخواه حالت‌ها. الگوریتم‌های پالایش افراز معمولاً از یک افراز اولیه شروع می‌کنند و هر بلوکی را که اعضایش نتوانند گام‌های یکدیگر را پاسخ دهند می‌شکنند. این فرایند تا رسیدن به افزازی پایدار ادامه می‌یابد [۱۷، ۱۰، ۳].

این زنجیره مفهومی جایگاه مسئله را روشن می‌کند: واریسی مدل با رشد فضای حالت محدود می‌شود؛ سامانه انتقال زمان‌دار افزون بر کنش، گذر زمان را نیز ثبت می‌کند؛ و شبیه‌سازی دوسویه ضعیف زمان‌دار معیاری فراهم می‌کند تا جزئیات داخلی پنهان شوند، درحالی‌که رفتار زمانی قابل مشاهده حفظ می‌شود [۳، ۵، ۲۶]. فصل‌های بعد مشخص می‌کنند مدل ورودی این پروژه دقیقاً چه ساختاری دارد، تعریف بالا چگونه در الگوریتم به کار رفته است و کاهش حاصل با چه شواهدی ارزیابی می‌شود.

فصل ۳

روش طراحی و پیاده‌سازی

۱.۳ مقدمه

هدف این پروژه ساخت ابزاری است که فضای حالت زمان‌دار صادرشده از افرا را دریافت کند، تعامل‌های خارج از مجموعه مورد توجه کاربر را پنهان سازد، حالت‌ها را بر اساس شبیه‌سازی دوسویه ضعیف زمان‌دار افراز کند و سامانه خارج‌قسمتی حاصل را تحویل دهد. ورودی الگوریتم یک سامانه انتقال متناهی است، نه کد منبع ربکا؛ بنابراین، تولید فضای حالت همچنان بر عهده زنجیره ابزار ربکا و افراست. پیاده‌سازی حاصل با نام awtr در مخزن پروژه نگهداری می‌شود [۱۵].

روش انجام کار ترکیبی از طراحی الگوریتم، مهندسی نرم‌افزار و ارزیابی تجربی است. ابتدا قرارداد واقعی فایل خروجی افرا از روی تولیدکننده و خواننده‌های رسمی آن استخراج شد. سپس ساختار الگوریتم بر پایه پالایش افراز—همان ایده‌ای که گروه و واندراخر برای شبیه‌سازی دوسویه انشعابی به کار بردند [۱۰]—برای حالت زمان‌دار تطبیق داده شد، و نحوه برخورد با تأخیرها مطابق ویژگی‌های تعریف ۲.۶.۲ تعیین شد. پس از ساخت افراز، سامانه انتقال خارج‌قسمتی تولید و با واری‌گری مستقل بررسی شد. در پایان، رفتار ابزار با آزمون‌های واحد، آزمون‌های انتها به انتها^۱ و ورودی‌های ارزیابی سنجیده شد. این فصل روش و تصمیم‌های طراحی را شرح می‌دهد و فصل ۴ نتایج ثبت‌شده را گزارش می‌کند.

¹End-to-End Testing

۲.۳ صورت مسئله، ورودی و خروجی

فرض کنید سامانه انتقال زمان‌دار متناهی پس از خواندن فایل ورودی به صورت

$$T = (S, s_0, Act \cup \{\tau\} \cup \mathbb{N}_{>0}, \rightarrow)$$

نمایش داده شود. S مجموعه حالت‌ها، s_0 حالت آغازین، Act مجموعه تعامل‌های نام‌دار و $d \in \mathbb{N}_{>0}$ برچسب گذر دقیقاً d واحد زمان است. کاربر در هر اجرا زیرمجموعه‌ای مانند $O \subseteq Act$ را به عنوان تعامل‌های قابل مشاهده تعیین می‌کند. همه تعامل‌های دیگر به τ نگاشت می‌شوند و سپس افراز پایدار متناظر با تعریف ۳.۸.۲ محاسبه می‌شود.

خروجی اصلی فقط یک پاسخ بله یا خیر نیست. ابزار باید سه شیء مرتبط را تولید کند: نگاشت هر حالت ورودی به رده هم‌ارزی، سامانه انتقال خارج‌قسمتی، و رابطه‌ای که نشان دهد مدل ورودی و خروجی مطابق تعریف انتخاب‌شده رفتار یکسان دارند. علاوه بر این، برای تکرارپذیری هر اجرا، مجموعه تعامل‌های قابل مشاهده، هش ورودی، نسخه ابزار، شناسه کامیت، زمان اجرا و اندازه‌های پیش و پس از کاهش ثبت می‌شوند [۱۵].

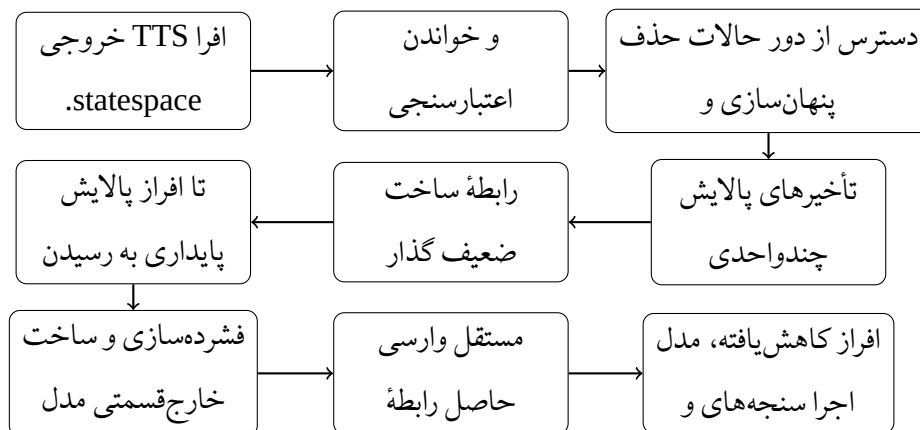
دامنه روش به زمان گسسته و تأخیرهای صحیح نامنفی محدود است. ورودی‌های زمان چگال، معنای FTS، واریسی شبیه‌سازی دوسویه قوی و کامپایل مستقیم کد ربکا در این پروژه پیاده‌سازی نشده‌اند. همچنین، هدف ساخت مدلی کوچک‌تر و واریسی شده است؛ مینیمال یا یکتا بودن مدل خروجی ادعا نمی‌شود.

۳.۳ جریان پردازش

شکل ۱.۳ مسیر داده را از فایل افرا تا مصنوعات خروجی نشان می‌دهد. مرز میان خواندن فایل و هسته الگوریتم صریح نگه داشته شده است؛ از این رو، تغییر منبع داده، منطق پنهان‌سازی^۲، پالایش افراز یا ساخت خارج‌قسمت را تغییر نمی‌دهد.

این تفکیک دو فایده دارد. نخست، هر مرحله قرارداد مشخصی دارد و می‌توان آن را جداگانه آزمود. دوم، واریسی‌گر خروجی از پیاده‌سازی پالایش افراز استفاده مجدد نمی‌کند؛ در نتیجه، خطای مشترک در محاسبه رابطه

²Hiding



شکل ۱.۳: جریان پردازش در ابزار کاهش فضای حالت زمان‌دار.

ضعیف و واریسی آن به‌سادگی پنهان نمی‌شود.

۴.۳ خواندن خروجی افرا

۱.۴.۳ قرارداد فایل TTS

ساختار فایل statespace با بررسی تولیدکننده رسمی RMC، طرح داده افرا و دو خواننده رسمی فضای حالت تعیین شد [۱۹، ۲۱، ۲۵، ۲۴]. قواعدی که مستقیماً بر پیاده‌سازی اثر گذاشتند در جدول ۱.۳ آمده‌اند.

جدول ۱.۳: بخش مورد استفاده از قرارداد خروجی فضای حالت افرا

تفسیر در ابزار	جزء فایل
شناسه حالت به‌صورت رشته‌ای و بدون تفسیر ساختار داخلی آن نگهداری می‌شود.	state/@id
حالت آغازین است؛ گزینه initial-state- امکان جایگزینی صریح آن را فراهم می‌کند.	state نخستین عنصر
ویژگی title هویت پایه تعامل است؛ صورت owner.title نیز برای رفع ابهام قابل انتخاب است.	messageserver
مدت سپری‌شده است و باید یک عدد صحیح نامنفی باشد.	time/@value
برای ردیابی نگه داشته می‌شوند، اما در مدت گذار دخالت داده نمی‌شوند.	shift و executionTime

یک ناسازگاری ظاهری در خروجی واقعی وجود دارد: تولیدکننده RMC عنصر ریشه `transitionsystem` را باز می‌کند، اما برچسب پایانی آن را نمی‌نویسد. بنابراین، فایل واقعی از دید یک تجزیه‌گر سخت‌گیر، XML خوش‌ساخت نیست. خواننده این پروژۀ انتهایی فایل را بررسی می‌کند و فقط در صورت نبودن برچسب پایانی، آن را به جریان ورودی اضافه می‌کند. به این ترتیب فایل بدون ویرایش دستی مصرف می‌شود و فایل‌های دارای برچسب پایانی نیز پذیرفته می‌شوند [۲۱، ۲۴، ۱۵].

تنها خروجی‌های TTS برای کاهش زمانی مناسب‌اند؛ زیرا در این حالت، گذر زمان به صورت عناصر مستقل `time` ثبت می‌شود. در FTTS زمان در گذارهای پیام ادغام می‌شود و یال تأخیر مستقلی برای الگوریتم حاضر وجود ندارد. خواننده همچنین شناسۀ تکراری حالت، مقصد یا مبدأ تعریف نشده، زمان منفی یا اعشاری، پیام فاقد ویژگی‌های لازم، و وجود هم‌زمان برچسب پیام و زمان روی یک گذار را رد می‌کند. پردازش DTD و موجودیت خارجی نیز غیرفعال شده است.

۲.۴.۳ مرز منبع داده

خواننده فایل یکی از پیاده‌سازی‌های رابط `TransitionSystemSource` است. این رابط تنها یک عمل برای ساخت نمایش خام سامانه دارد. نمایش خام، نام فرستنده، مالک و عنوان پیام و نیز اطلاعات زمانی جانبی را حفظ می‌کند. سپس مرحله جداگانه‌ای آن را به سامانه انتقال مورد استفاده الگوریتم تبدیل می‌کند. یک منبع درون حافظه‌ای نیز همین رابط را پیاده‌سازی می‌کند و آزمون قرارداد نشان می‌دهد هر دو منبع به مدل دامنه و نتیجه کاهش یکسان می‌رسند. این مرز امکان افزودن یک آداپتور درون‌فرایندی برای افرا را فراهم می‌کند، بی‌آنکه هسته کاهش به کلاس‌های رابط کاربری یا XML وابسته شود [۱۵].

۵.۳ انتخاب مشاهده‌ها و پیش‌پردازش

مجموعه تعامل‌های قابل مشاهده در زمان اجرا و با گزینه `observable`—تعیین می‌شود. یک نام ساده مانند `getSense` با عنوان پیام در هر مالک تطبیق داده می‌شود؛ صورت مقید مانند `controller.getSense` فقط همان مالک را می‌پذیرد. تطبیق به بزرگی و کوچکی حروف حساس است. این تصمیم از ادغام ناخواسته دو شناسه متمایز جلوگیری می‌کند و با هویت ثبت‌شده در خروجی افرا سازگار است [۲۴، ۲۵].

اگر α یک تعامل نام دار و O مجموعه انتخابی کاربر باشد، تابع پنهان سازی به صورت زیر عمل می کند:

$$h_O(\alpha) = \begin{cases} \alpha & \alpha \in O, \\ \tau & \alpha \notin O. \end{cases}$$

گذر بدون برچسب و گذر زمان صفر نیز به τ تبدیل می شوند. عنوان هایی که افرا با پیشوند τ به عنوان ادامه اجرای یک پیام ثبت کرده است، با یک نام ساده قابل مشاهده نمی شوند. پس از پنهان سازی، فقط زیرسامانه قابل دسترس از حالت آغازین نگه داشته می شود. ابزار نام های قابل مشاهده ای را که با هیچ گذار ورودی تطبیق ندارند نیز گزارش می کند؛ زیرا یک خطای تایپی در این گزینه می تواند ناخواسته همه تعامل ها را پنهان کند.

۶.۳ چرا یال های تأخیر به گام های یک واحدی شکسته می شوند

مهم ترین تصمیم معنایی این پروژه، شکستن هر یال تأخیر با مدت d به d یال یک واحدی است. این تصمیم یک بهینه سازی یا سلیقه پیاده سازی نیست؛ بدون آن، الگوریتم افراز نمی تواند تعریف ۳.۸.۲ را روی خروجی افرا محاسبه کند. دلیل آن به فاصله میان دو چیز بازمی گردد: سامانه انتقال زمان داری که مدل را توصیف می کند، و گراف متناهی ای که افرا از آن ذخیره می کند.

۱.۶.۳ مسئله: حالت هایی که در گراف ذخیره شده شریکی ندارند

دو سامانه زیر را در نظر بگیرید که در آن ها a تنها کنش قابل مشاهده است:

$$\begin{aligned} A : \quad & q_0 \xrightarrow{2} q_1 \xrightarrow{3} q_2 \xrightarrow{a} q_3, \\ B : \quad & p_0 \xrightarrow{5} p_1 \xrightarrow{a} p_2. \end{aligned}$$

هر دو سامانه دقیقاً یک رفتار قابل مشاهده دارند: پس از گذشت پنج واحد زمان، کنش a رخ می دهد. طبق تعریف ۳.۸.۲ این دو سامانه شبیه سازی دوسویه ضعیف زمان دار دارند، زیرا رابطه گذار ضعیف بر اساس وجود یک اجرا

با مدت کل معین تعریف شده است، نه بر اساس وجود یک یال منفرد؛ در A اجرایی از q_0 به q_2 با مدت کل ۵ و بدون کنش قابل مشاهده وجود دارد، پس $q_2 \xrightarrow{5} q_0$ برقرار است و $p_0 \xrightarrow{5} p_1$ نیز. اکنون فرض کنید بخواهیم چنین رابطه‌ای را فقط میان حالت‌های نوشته‌شده در همین دو گراف بسازیم. حالت q_1 ، یعنی لحظه «دو واحد گذشته است»، در A صریحاً وجود دارد؛ زیرا $q_1 \xrightarrow{2} q_0$ برقرار است، رابطه باید برای q_1 شریکی در B بیابد. اما در B هیچ حالتی متناظر با لحظه «دو واحد گذشته است» نوشته نشده است: تنها گذار تأخیری موجود، یک یال پنج‌واحدی است و $p_0 \xrightarrow{5} p'$ برای هیچ p' از حالت‌های $\{p_0, p_1, p_2\}$ برقرار نیست. در نتیجه q_1 با هیچ حالتی جفت نمی‌شود و ساخت رابطه شکست می‌خورد—درحالی‌که دو سامانه طبق تعریف هم‌ارزند.

۲.۶.۳ راه‌حل: استفاده از ویژگی پیوستگی

این تناقض ظاهری از یک اشتباه در سطح تحلیل می‌آید. مطابق تعریف ۲.۶.۲، رابطه گذار یک سامانه انتقال زمان‌دار ویژگی پیوستگی دارد: اگر $p_0 \xrightarrow{5} p_1$ و $p_0 \xrightarrow{3+2=5} p'$ آنگاه پیکربندی p' وجود دارد که

$$p_0 \xrightarrow{2} p' \xrightarrow{3} p_1.$$

یعنی لحظه میانی «دو واحد گذشته است» در سامانه انتقال زمان‌دار B حتماً وجود دارد؛ آنچه وجود ندارد، نمایش صریح آن در فایل خروجی افراست. افرا فقط تعدادی یال تأخیر با مدت‌های معین را ثبت می‌کند و پیکربندی‌های میانی را نمی‌نویسد، زیرا برای تحلیل‌های متداول خود به آن‌ها نیاز ندارد.

برای آنکه الگوریتم افراز بتواند روی همین گراف متناهی کار کند، باید آن پیکربندی‌های میانی را که تعریف به آن‌ها نیاز دارد، صریحاً بسازیم. در حالت کلی این کار شدنی نیست: در زمان چگال میان هر دو لحظه بی‌نهایت لحظه دیگر وجود دارد. اما در این پروژه، همان‌طور که در بخش پیش‌تر آمد، دامنه زمان $\mathbb{T} = \mathbb{N}$ است و همه مدت‌های ثبت‌شده اعداد صحیح‌اند. پس کوچک‌ترین واحد زمان وجود دارد و تنها لحظه‌های میانی‌ای که می‌توانند مبدأ یا مقصد یک گذار باشند، لحظه‌های صحیح‌اند. بنابراین کافی است هر یال $s \xrightarrow{d} t$ با $d > 1$ پیش از اجرای الگوریتم به زنجیره زیر بازنویسی شود:

$$s \xrightarrow{1} v_1 \xrightarrow{1} v_2 \xrightarrow{1} \dots \xrightarrow{1} v_{d-1} \xrightarrow{1} t.$$

حالت‌های v_i همان پیکربندی‌های میانی‌اند که ویژگی پیوستگی وجودشان را تضمین می‌کند. شناسه‌های این حالت‌ها تازه و یکتا انتخاب می‌شوند تا با حالت‌های اظهارشده در ورودی اشتباه نشوند. پس از این بازنویسی، مثال بالا به‌درستی حل می‌شود: در B حالت میانی متناظر با q_1 ساخته شده و رابطه کامل می‌شود. این بازنویسی یک سود الگوریتمی دیگر هم دارد. پس از آن، تنها برچسب زمانی موجود در مدل، تأخیر یک‌واحدی است و رسیدن ضعیف با مدت d برابر با ترکیب d باره رسیدن ضعیف یک‌واحدی است. در نتیجه مجموعه برچسب‌هایی که پالایش افراز باید بررسی کند متناهی و کوچک می‌ماند و الگوریتم لازم نیست همه مدت‌های قابل دستیابی را جداگانه تولید کند. به بیان دیگر، گذر زمان و اجرای کنش پس از این بازنویسی برای الگوریتم افراز یکسان به نظر می‌رسند و هر دو با یک سازوکار بررسی می‌شوند.

۳.۶.۳ هزینه و سقف کنترلی

هزینه این روش افزودن

$$\sum_{e \in E_{time}} (d_e - 1)$$

حالت میانی است؛ یعنی اندازه کار مستقیماً به بزرگی ثابت‌های زمانی مدل وابسته است. ابزار برای تعداد حالت‌های افزوده‌شده سقفی قابل تنظیم دارد و در صورت عبور از آن، به‌جای رشد کنترل‌نشده حافظه، اجرا را با پیام خطا متوقف می‌کند. این راه‌حل برای مدل‌های ارزیابی حاضر کافی است، اما در مدل‌هایی با ثابت‌های زمانی بزرگ می‌تواند عامل اصلی مصرف حافظه باشد. فصل ۵ راهی را مطرح می‌کند که همین نتیجه را بدون شکستن یال‌ها به دست می‌آورد.

۴.۶.۳ دو حالت اجرایی و تفاوت معنایی آن‌ها

پیاده‌سازی این تصمیم را پنهان نمی‌کند و آن را به‌صورت گزینه time-semantics- در دسترس می‌گذارد:

- حالت unit، که پیش‌فرض است و همه نتایج این پایان‌نامه بر مبنای آن گزارش می‌شوند. در این حالت یال‌های تأخیر به‌شرح بالا شکسته می‌شوند و آنچه محاسبه می‌شود دقیقاً افراز متناظر با تعریف ۳.۸.۲ است.

- حالت strict، که یال‌های تأخیر را دست‌نخورده نگه می‌دارد و فقط مدت‌های ثبت‌شده در فایل ورودی را به‌عنوان برچسب زمانی می‌پذیرد.

باید تصریح کرد که نتیجه حالت strict یک شبیه‌سازی دوسویه ضعیف زمان‌دار به معنای تعریف ۳.۸.۲ نیست و به‌کاربردن این نام برای آن گمراه‌کننده است. آن حالت در عمل رابطه‌گذار ورودی را همان‌گونه که ذخیره شده است می‌پذیرد و ویژگی پیوستگی تعریف ۲.۶.۲ را به کار نمی‌گیرد؛ یعنی روی ساختاری کار می‌کند که ویژگی‌های استاندارد یک سامانه انتقال زمان‌دار را ندارد. دلیل نگه‌داشتن آن صرفاً تشخیصی است: با آن می‌توان نشان داد که کدام نتیجه‌ها واقعاً به شکستن یال‌ها وابسته‌اند و بنابراین تصمیم معنایی پروژه در کد پنهان نمانده و قابل بازیابی است. بخش ۷.۴ این تفاوت را روی نمونه‌های واقعی اندازه می‌گیرد. هدف و قرارداد معنایی پروژه در همه حال حالت unit است.

خانواده mood در فصل ۴ همین وضعیت را در یک مدل واقعی ربکا نشان می‌دهد: در یکی از دو مدل، یک انتظار پنج‌واحدی با یک یال ثبت شده و در دیگری همان انتظار به‌صورت دو واحد، یک گام داخلی و سه واحد ظاهر شده است.

۷.۳ ساخت رابطه‌گذار ضعیف

پس از پالایش زمانی، برای هر حالت ابتدا بستار τ با یک پیمایش عمقی محاسبه می‌شود. سپس برای هر برچسب قابل مشاهده و برچسب تأخیر واحد، مجموعه مقصدهای $\tau^* \ell \tau^*$ ساخته می‌شود. بستار τ نیز به‌عنوان حرکت تهی در پالایش شرکت می‌کند؛ این همان حالت مدت صفر در تعریف ۲.۸.۲ است. حذف این حالت می‌تواند شاخه داخلی منتهی به بن‌بست را از دید الگوریتم پنهان کند.

در پیاده‌سازی، حالت‌ها یک‌بار شماره‌گذاری و مجموعه مقصدها با BitSet نگهداری می‌شوند. بنابراین، محاسبه اجتماع مقصدها و مقایسه امضاها از عملیات بیتی استفاده می‌کند. ترتیب حالت‌ها از ورودی و ترتیب برچسب‌ها به‌صورت مرتب‌شده حفظ می‌شود تا اجرای دوباره روی ورودی یکسان مصنوعات متنی یکسانی تولید کند [۱۵].

۸.۳ ایده‌مبنا: پالایش افراز برای شبیه‌سازی دوسویه انشعابی

الگوریتم کاهشی که در این پروژه پیاده‌سازی شده است، الگوریتم تازه‌ای از پایه نیست. ساختار آن از الگوریتم شناخته‌شده گروته و واندراخر برای محاسبه شبیه‌سازی دوسویه انشعابی روی سامانه‌های انتقال برچسب‌دار گرفته شده و برای حالت زمان‌دار تطبیق داده شده است [۱۰]. پیش از شرح الگوریتم پروژه، لازم است ایده آن الگوریتم با دقت کافی بیان شود تا روشن شود چه چیزی وام گرفته شده و چه چیزی تغییر کرده است.

۱.۸.۳ مسئله درشت‌ترین افراز

نقطه شروع این خانواده از الگوریتم‌ها مسئله درشت‌ترین افراز است که پیچ و تارجان صورت‌بندی کردند [۱۷]. به جای آنکه همه رابطه‌های ممکن میان حالت‌ها جست‌وجو شوند، از یک افراز درشت آغاز می‌شود و تا رسیدن به پایداری شکسته می‌شود. اصطلاح کلیدی، پایداری است: افراز P نسبت به بلوک B' و برچسب a پایدار است، هرگاه برای هر بلوک $B \in P$ ، یا همه اعضای B بتوانند با کنش a به B' برسند، یا هیچ‌کدام نتوانند. اگر بلوکی این شرط را نقض کند، آن بلوک به دو بخش «می‌رسند» و «نمی‌رسند» شکسته می‌شود و زوج (B', a) را شکننده می‌نامیم.

سه نکته این روش را کارآمد می‌کند. نخست، هر شکستن، افراز را ریزتر می‌کند و چون تعداد حالت‌ها متناهی است، فرایند حتماً پایان می‌یابد. دوم، بلوک‌های شکسته‌شده هرگز دوباره ادغام نمی‌شوند، پس افراز نهایی یکتاست. سوم، افراز نهایی درشت‌ترین افراز پایدار است؛ یعنی هیچ دو حالتی را که واقعاً تمایزپذیرند در یک رده نگه نمی‌دارد و هیچ دو حالت تمایزناپذیری را نیز بیهوده جدا نمی‌کند. برای شبیه‌سازی دوسویه قوی، همین افراز دقیقاً رده‌های هم‌ارزی مطلوب است [۱۷، ۳].

۲.۸.۳ آنچه گروته و واندراخر افزودند

اگر مدل کنش داخلی داشته باشد، شرط بالا بیش از اندازه سخت‌گیر است. یک گام τ نباید صرفاً به دلیل وجودش باعث تمایز دو حالت شود. گروته و واندراخر این مشکل را با مفهوم گذار τ بی‌اثر حل کردند: گذار $t \xrightarrow{\tau} s$ در افراز جاری بی‌اثر است، هرگاه s و t در یک بلوک باشند. چنین گامی رده رفتاری را تغییر نداده است و

الگوریتم آن را نادیده می‌گیرد؛ این همان بند اول تعریف ۲.۷.۲ است [۳۲، ۱۰].

با این مفهوم، شرط پایداری بازنویسی می‌شود: بلوک B نسبت به زوج (B', a) پایدار است، هرگاه یا همه اعضای B بتوانند با دنباله‌ای از گذارهای τ بی‌اثر یعنی بدون خروج از B و سپس یک گذار a به B' برسند، یا هیچ‌کدام نتوانند. مسیرهای داخلی مجاز در پاسخ، همان مسیرهایی‌اند که رده رفتاری را ترک نمی‌کنند. هر بار که بلوکی شکسته می‌شود، ممکن است گذارهایی که تا پیش از آن بی‌اثر بودند دیگر بی‌اثر نباشند؛ از این رو الگوریتم باید بلوک‌های متأثر را دوباره بررسی کند. این تکرار تا جایی ادامه می‌یابد که هیچ زوج شکننده‌ای باقی نماند. الگوریتم حاصل پیچیدگی زمانی $O(n(n+m))$ و پیچیدگی فضایی $O(n+m)$ دارد که n تعداد حالت‌ها و m تعداد گذارهاست [۱۰]؛ نسخه‌های بعدی این مرز را به $O(m \log n)$ رسانده‌اند [۹].

۳.۸.۳ آنچه در این پروژه تغییر کرده است

الگوریتم این پروژه سه عنصر بالا را نگه می‌دارد: شروع از یک افراز درشت، شکستن بلوک‌ها بر اساس بلوک‌های قابل دستیابی، و ادامه تا رسیدن به پایداری. سه تفاوت اصلی نیز دارد:

۱. هم‌ارزی هدف. معیار این پروژه شبیه‌سازی دوسویه ضعیف زمان‌دار است، نه انشعابی. بنابراین شرط عضویت حالت‌های میانی مسیر داخلی در همان رده یعنی شرط $(p, q_1) \in R$ در تعریف ۲.۷.۲ — برداشته می‌شود و به جای مسیرهای τ بی‌اثر، بستار کامل τ به کار می‌رود. این تصمیم از تعریف ۳.۸.۲ می‌آید که مبنای پذیرش پروژه است، نه از انتخاب الگوریتمی.

۲. حضور زمان در مجموعه برچسب‌ها. در مدل زمان‌دار، برچسب یک گذار می‌تواند مدت نیز باشد. پس از بازنویسی بخش ۶.۳، تنها برچسب زمانی موجود تأخیر یک‌واحدی است؛ در نتیجه زمان نیز مانند یک کنش معمولی در همان سازوکار پایداری شرکت می‌کند و لازم نیست برای هر مدت ممکن، یک برچسب جداگانه ساخته شود.

۳. شکستن مبتنی بر امضا. به جای انتخاب یک زوج شکننده در هر دور، این پیاده‌سازی برای هر حالت یک امضا از مجموعه بلوک‌های قابل دستیابی می‌سازد و همه بلوک‌ها را در یک گذر بر اساس امضا می‌شکند. این صورت ساده‌تر، همان افراز پایدار را می‌دهد اما به مرز پیچیدگی بهینه منابع بالا نمی‌رسد؛ برای اندازه مدل‌های این پروژه کافی است و بخش ۵ بهبود آن را در کارهای آینده مطرح می‌کند.

۹.۳ پالایش افراز

الگوریتم زیر همان طرح بخش ۸.۳ است که با سه تغییر یادشده برای شبیه‌سازی دوسویه ضعیف زمان‌دار بازنویسی شده است. الگوریتم با افراز یک‌بلوکی $P_* = \{S\}$ آغاز می‌شود. برای افراز جاری P و برچسب ℓ ، مجموعه بلوک‌های قابل دستیابی از s برابر است با

$$R_P(s, \ell) = \{[t]_P \mid s \xRightarrow{\ell}_W t\}.$$

امضای هر حالت از بلوک فعلی آن، مجموعه بلوک‌های قابل دستیابی برای تمام برچسب‌ها، و بلوک‌های موجود در بستر τ ساخته می‌شود:

$$\text{Sig}_P(s) = \left([s]_P, (R_P(s, \ell))_{\ell \in L}, \{[t]_P \mid t \in \tau^*(s)\} \right).$$

حالت‌های یک بلوک که امضای متفاوت دارند از یکدیگر جدا می‌شوند. وجود $[s]_P$ در امضا تضمین می‌کند دوره‌های بعدی فقط بلوک‌های موجود را بشکنند و دو بلوک جداشده دوباره ادغام نشوند. فرایند زمانی پایان می‌یابد که هیچ امضای تازه‌ای باعث شکستن بلوک نشود. افراز حاصل پایدار است و رده‌های شبیه‌سازی دوسویه ضعیف زمان‌دار را برای معنای ورودی محاسبه می‌کند.

الگوریتم‌های ۱.۳ تا ۴.۳ شبه‌کد اصلی این فرایند را در چهار مرحله نشان می‌دهند. جداسازی این مراحل روشن می‌کند که رابطه ضعیف چگونه از گذارهای مستقیم ساخته می‌شود و افراز در هر دور دقیقاً بر چه اساسی شکسته می‌شود. در این الگوریتم‌ها، $C_\tau(s)$ بستر کنش داخلی از حالت s ، L مجموعه برچسب‌های غیر از τ و W رابطه گذار ضعیف است. در حالت پیش‌فرض پیاده‌سازی، پالایش واحدی پیش از اجرای این مراحل انجام می‌شود و تنها برچسب زمانی موجود در L برابر یک واحد است؛ در خوانش سخت‌گیرانه، L مدت‌های ثبت‌شده روی یال‌های ورودی را نیز در بر می‌گیرد.

الگوریتم ۱.۳ بستر داخلی یک حالت را محاسبه می‌کند. این بستر شامل همه حالت‌هایی است که با صفر یا چند گذار داخلی دسترس‌پذیرند. افزودن خود s به مجموعه آغازین، حرکت داخلی تهی را نیز پوشش می‌دهد. مجموعه visited مانع بررسی دوباره حالت‌ها می‌شود و پایان پیمایش را در حضور چرخه‌های τ تضمین می‌کند. الگوریتم ۲.۳ یک برچسب غیر از τ را میان دو بستر داخلی قرار می‌دهد. اگر ℓ یک کنش قابل مشاهده

الگوریتم ۱.۳ Tau Closure

Require: A transition system $T = (S, L \cup \{\tau\}, \rightarrow)$ and a state $s \in S$

Ensure: All states reachable from s through zero or more τ transitions

```

1:  $visited \leftarrow \{s\}$ 
2:  $stack \leftarrow [s]$ 
3: while  $stack$  is not empty do
4:    $x \leftarrow \text{pop}(stack)$ 
5:   for each transition  $(x, \tau, y) \in \rightarrow$  do
6:     if  $y \notin visited$  then
7:        $visited \leftarrow visited \cup \{y\}$ 
8:        $\text{push}(stack, y)$ 
9:     end if
10:  end for
11: end while
12: return  $visited$ 

```

باشد، خروجی همان $\tau^* \ell \tau^*$ است؛ اگر ℓ یک برچسب زمانی باشد، خروجی مقصدهای گذار ضعیف با همان مدت را می‌دهد. در حالت unit ، تکرار این رابطه یک‌واحدی مدت‌های بزرگ‌تر را می‌سازد.

الگوریتم ۳.۳ دو محاسبه قبلی را برای همه حالت‌ها و برچسب‌ها انجام می‌دهد. خروجی آن علاوه بر W ، بستارهای داخلی را نیز نگه می‌دارد؛ زیرا $C_\tau(s)$ حالت مدت صفر تعریف ۲.۸.۲ است و در پالایش افراز باید مانند دیگر حرکت‌های ضعیف بررسی شود.

الگوریتم ۴.۳ با قرار دادن همه حالت‌ها در یک بلوک آغاز می‌شود. در هر دور، برای هر حالت مجموعه بلوک‌های قابل دسترس با هر برچسب و مجموعه بلوک‌های موجود در بستار داخلی محاسبه می‌شود. دو حالت تنها زمانی در یک بلوک می‌مانند که بلوک فعلی و همه این مجموعه‌ها برای آن‌ها یکسان باشند. مقایسه آرایه nextBlock با block شرط توقف را به صورت صریح نشان می‌دهد.

پیاده‌سازی مبتنی بر امضا در هر دور، برای هر حالت و هر برچسب مجموعه بلوک‌های مقصد را بررسی می‌کند و حداکثر به تعداد حالت‌ها دور دارد. الگوریتم‌های پالایش پیشرفته‌تری با کران مجانبی بهتر وجود دارند، اما روش حاضر با شبه‌کد پروژه تطابق مستقیم دارد و برای بزرگ‌ترین ورودی موجود، که ۴۲ حالت پیش از پالایش زمانی دارد، کافی بوده است. فصل ۴ از این اندازه فراتر ادعای مقیاس‌پذیری نمی‌کند.

 الگوریتم ۲.۳ Weak Successors for One Label

Require: A transition system T , tau closures C_τ , a state s , and a label $\ell \neq \tau$

Ensure: The targets of the weak step $s \xRightarrow{\ell}_W t$

```

1:  $targets \leftarrow \emptyset$ 
2: for each state  $x \in C_\tau(s)$  do
3:   for each transition  $(x, \ell, y) \in \rightarrow$  do
4:     for each state  $t \in C_\tau(y)$  do
5:        $targets \leftarrow targets \cup \{t\}$ 
6:     end for
7:   end for
8: end for
9: return  $targets$ 
  
```

 الگوریتم ۳.۳ Weak Transition Relation

Require: A timed transition system $T = (S, Act \cup \{\tau\} \cup D, \rightarrow)$

Ensure: Tau closures C_τ , non-tau labels L , and weak relation W

```

1: for each state  $s \in S$  do
2:    $C_\tau(s) \leftarrow \text{TauClosure}(T, s)$ 
3: end for
4:  $L \leftarrow \{\ell \mid (x, \ell, y) \in \rightarrow \wedge \ell \neq \tau\}$ 
5:  $W \leftarrow \emptyset$ 
6: for each state  $s \in S$  do
7:   for each label  $\ell \in L$  do
8:      $A \leftarrow \text{WeakSuccessors}(T, C_\tau, s, \ell)$ 
9:     for each state  $t \in A$  do
10:       $W \leftarrow W \cup \{(s, \ell, t)\}$ 
11:    end for
12:   end for
13: end for
14: return  $(C_\tau, L, W)$ 
  
```

الگوریتم ۴.۳ Stable Partition Refinement

Require: States S , tau closures C_τ , labels L , and weak relation W

Ensure: The coarsest stable partition P

```

1: for each state  $s \in S$  do
2:    $block[s] \leftarrow 0$ 
3: end for
4: repeat
5:    $signatureIds \leftarrow$  an empty ordered map
6:   for each state  $s \in S$  do
7:     for each label  $\ell \in L$  do
8:        $R_\ell(s) \leftarrow \{block[t] \mid (s, \ell, t) \in W\}$ 
9:     end for
10:     $R_{\tau^*}(s) \leftarrow \{block[t] \mid t \in C_\tau(s)\}$ 
11:     $\sigma(s) \leftarrow (block[s], (R_\ell(s))_{\ell \in L}, R_{\tau^*}(s))$ 
12:    if  $\sigma(s) \notin signatureIds$  then
13:      Assign the next unused block id to  $signatureIds[\sigma(s)]$ .
14:    end if
15:     $nextBlock[s] \leftarrow signatureIds[\sigma(s)]$ 
16:  end for
17:   $changed \leftarrow (nextBlock \neq block)$ 
18:   $block \leftarrow nextBlock$ 
19: until  $changed = false$ 
20:  $P \leftarrow \{\{s \in S \mid block[s] = i\} \mid i \text{ is an assigned id}\}$ 
21: return  $P$ 

```

۱۰.۳ ساخت مدل خارج قسمتی و فشرده سازی زمان

در ساخت خارج قسمت، هر رده هم ارزی به یک حالت تبدیل می شود و اگر یالی با برچسب ℓ از عضوی از رده B به عضوی از رده B' وجود داشته باشد، یال $B \xrightarrow{\ell} B'$ ساخته می شود. خودحلقه τ درون یک رده حذف می شود؛ زیرا چیزی به بستار داخلی آن رده اضافه نمی کند.

خارج قسمت مستقیم سامانه پالایش شده فقط یال های یک واحدی دارد. در نتیجه، یک انتظار ده واحدی می تواند چندین رده میانی ایجاد کند و حتی مدل ظاهراً کاهش یافته را از ورودی بزرگ تر سازد. برای جلوگیری از این مسئله، زنجیره پیشینه ای از رده ها که تنها یک یال زمانی ورودی و یک یال زمانی خروجی دارند، حالت آغازین نیستند و هیچ انتخاب یا کنش قابل مشاهده ای ندارند، به یک یال با مدت مجموع تبدیل می شود. شرط ساختاری سخت گیرانه این مرحله مانع حذف نقطه ای می شود که محل انتخاب رفتاری است.

به همین دلیل، تعداد رده های افراز لزوماً با تعداد حالت های خروجی یکسان نیست. هر رده یا به یک حالت خارج قسمتی تبدیل می شود یا موقعیت آن روی یک یال تأخیر فشرده شده ثبت می شود. فایل partition.json این نداشت را نگه می دارد. همچنین، اطلاعات لازم برای بازکردن دوباره یال های چند واحدی ذخیره می شود تا واریسی گر بتواند دقیقاً مدل تحویلی را با سامانه پالایش شده مقایسه کند.

۱۱.۳ واریسی مستقل خروجی

واریسی گر رابطه میان حالت های سامانه پالایش شده و مدل خارج قسمتی را مستقیماً در برابر تعریف شبیه سازی دوسویه ضعیف زمان دار می سنجد. برای هر زوج مرتبط، همه حرکت های تهی و همه گذارهای ضعیف یک طرف باید با گذار هم برچسب طرف دیگر پاسخ داده شوند و مقصدها نیز مرتبط بمانند؛ این بررسی در هر دو جهت انجام می شود. حالت های آغازین نیز باید در رابطه باشند.

واریسی گر از بستارهای محاسبه شده در کاهش دهنده استفاده نمی کند. برای نمونه، بستارها را با مجموعه های معمولی از نو می سازد؛ در حالی که کاهش دهنده مقصدها را در ساختار BitSet نگه می دارد. استقلال کامل منطقی ادعا نمی شود، زیرا هر دو مؤلفه تعریف معنایی مشترکی دارند. با این حال، جدایی کد باعث می شود خطای جزئی در شاخص گذاری یا عملیات بیتی کاهش دهنده، خودبه خود در واریسی گر تکرار نشود.

برای نشان دادن توان رد کردن، پنج تغییر مخرب روی مدل خارج قسمتی اعمال شده است: حذف یال، تغییر

برچسب، تغییر مدت، تغییر مقصد و افزودن یال. انتظار می‌رود واری‌گری همه این تغییرها را رد کند.

۱۲.۳ معماری نرم‌افزار و رابط اجرا

پیاده‌سازی با جاوای ۱۷ و Maven ساخته شده است. نسخه جاوا و نسخه‌های افزونه‌های ساخت با مخازن رسمی کامپایلر و واری‌گری مدل ربکا هم‌راستا شده‌اند [۲۰، ۲۲]. وابستگی زمان اجرا خارج از JDK وجود ندارد؛ خواندن فایل با StAX و تولید JSON با نویسنده‌ای کوچک و قطعی انجام می‌شود. این انتخاب احتمال تعارض کتابخانه‌ها را در صورت ادغام آینده با محیط مبتنی بر Eclipse افرا کاهش می‌دهد. بسته‌های اصلی نرم‌افزار مسئولیت‌های زیر را دارند:

- afra و source: خواندن فایل و تبدیل منبع داده به نمایش خام؛
- tts: نمایش تغییرناپذیر و اعتبارسنجی شده سامانه انتقال؛
- weak: پالایش زمانی، ساخت گذارهای ضعیف و پالایش افرا؛
- quotient: ساخت خارج‌قسمت و فشرده‌سازی زنجیره‌های زمانی؛
- verify: بررسی مستقل رابطه خروجی؛
- report: تولید مصنوعات و سنجه‌ها؛
- app: هماهنگ کردن مراحل و رابط خط فرمان.

رابط خط فرمان^۳ چهار دستور دارد: inspect برای شناخت محتوای ورودی، reduce برای کاهش یک سامانه، equivalent برای مقایسه حالت آغازین دو سامانه، و visualize برای تولید نمایش Graphviz. منطق اصلی در ReductionService قرار دارد و خط فرمان تنها گزینه‌ها و کد خروج را مدیریت می‌کند؛ بنابراین، همان جریان بدون ایجاد فرایند جداگانه از کد جاوا نیز قابل فراخوانی است. کد خروج صفر موفقیت، یک نتیجه معنایی منفی، دو ورودی نامعتبر و سه نقض یک ناوردای داخلی را نشان می‌دهد.

^۳Command-Line Interface

۱۳.۳ مصنوعات خروجی

جدول ۲.۳ نقش پنج فایل تولیدشده برای هر کاهش را نشان می‌دهد. هیچ‌یک از این فایل‌ها به‌تنهایی جای همهٔ بقیه را نمی‌گیرد؛ برای نمونه، نمایش سازگار با افرا برای مشاهده مناسب است، ولی عضویت رده‌ها و منشأ کامل یال‌ها را نگه نمی‌دارد.

جدول ۲.۳: مصنوعات تولیدشده برای هر اجرای کاهش

فایل	محتوا و کاربرد
reduced.json	نمایش اصلی و بدون اتلاف مدل کاهش‌یافته، شامل برچسب‌ها، اعضای رده‌ها و مجموعهٔ مشاهده‌ها.
partition.json	عضویت هر حالت ورودی در رده و موقعیت آن در مدل کاهش‌یافته.
reduced.statespace	نمای سازگار با ابزار نمایش فضای حالت افرا؛ این نما به‌دلیل محدودیت طرح داده اتلاف‌دار است.
reduced.dot	گراف DOT برای ترسیم ساختار مدل خارج‌قسمتی.
metrics.json	اندازه‌ها، نسبت‌های کاهش، زمان و حافظه، نتیجهٔ واریسی و اطلاعات ردیابی اجرا.

قطعی بودن ترتیب حالت‌ها، رده‌ها و یال‌ها موجب می‌شود چهار مصنوع ساختاری در دو اجرای یکسان، بابت‌به‌بایت برابر باشند. زمان اجرا و بیشینهٔ حافظهٔ طبیعتاً در metrics.json متغیرند. هش SHA-256 ورودی و شناسهٔ کامیت امکان اتصال هر نتیجه به فایل و نسخهٔ مولد آن را فراهم می‌کند.

۱۴.۳ طرح اعتبارسنجی و تکرار اجرا

اعتبارسنجی در سه سطح انجام شده است. در سطح معنایی، نمونه‌های کوچک با افراز مورد انتظار، بستر τ ، مشاهده‌ها، تأخیرها، بن‌بست و ورودی نامعتبر را می‌سنجند. در سطح مدل خارج‌قسمتی، کامل بودن افراز، معتبر بودن مقصد یال‌ها، رابطهٔ حالت آغازین و رد جهش‌های عمده بررسی می‌شود. در سطح انتها‌به‌انتها، فایل statespace. به برنامهٔ بسته‌بندی شده داده می‌شود و کد خروج، شمار حالت‌ها و محتوای همهٔ مصنوعات کنترل می‌شوند. جزئیات عددی این آزمون‌ها در فصل ۴ آمده است.

برای تکرار ساخت و آزمون نسخهٔ ثبت‌شدهٔ پروژه، فرمان‌های اصلی به‌شکل زیرند:

```

mvn clean package
python3 tests/e2e/run_e2e.py
python3 evaluation/run_evaluation.py

```

فرمان نخست نرم‌افزار و آزمون‌های واحد را می‌سازد، فرمان دوم رابط بسته‌بندی شده را مانند یک کاربر خارجی می‌آزماید و فرمان سوم فایل جمع‌بندی و سنجه‌های خام ارزیابی را از نو تولید می‌کند. ورودی‌ها، اسکریپت‌ها و نتایج ثبت شده در مخزن پروژه موجودند [۱۵].

۱۵.۳ جمع‌بندی

روش این پروژه یک زنجیره قابل ردیابی از خروجی TTS افرا تا مدل خارج‌قسمتی است. مجموعه مشاهده‌ها صریح است، تفاوت دو تفسیر تأخیر به گزینه و آزمون تبدیل شده، پالایش افراز از قرارداد ورودی جدا مانده و مدل تحویلی با پیاده‌سازی دیگری از شروط انتقال بررسی می‌شود. فصل بعد نشان می‌دهد این تصمیم‌ها روی نمونه‌های ارزیابی چه خروجی‌ای تولید کرده‌اند و شواهد تا چه اندازه از درستی و کارایی روش پشتیبانی می‌کنند.

فصل ۴

نتایج و اعتبارسنجی

۱.۴ مقدمه

این فصل خروجی ابزار را روی دو دسته ورودی گزارش می‌کند—چهار سامانه انتقال سنج‌ای و هفت فضای حالت تولیدشده از مدل‌های ربکا—و سپس شواهد اعتبارسنجی را بررسی می‌کند. پرسش اصلی این است که آیا روش فصل ۳ می‌تواند با حفظ رفتار ضعیف زمان‌دار، مدل کوچک‌تری تولید کند و آیا نمونه‌هایی که رفتار یکسان را با تقسیم‌بندی متفاوت زمان بیان می‌کنند به نتیجه‌ای سازگار می‌رسند. نتیجه‌ها از فایل‌های خام تولیدشده توسط اسکریپت ارزیابی گرفته شده‌اند؛ بنابراین، شمار حالت‌ها، گذارها و سنج‌های اجرا به یک ورودی و نسخه مشخص از برنامه قابل ردیابی اند [۱۵].

این ارزیابی برای استنتاج آماری یا تعیین روند رشد زمان اجرا طراحی نشده است. تعداد مدل‌ها کم است و هر اندازه‌گیری زمان تنها یک اجرای ثبت‌شده را نشان می‌دهد. از اعداد کارایی برای توصیف هزینه نمونه‌های موجود استفاده می‌شود، نه برای ادعای مقیاس‌پذیری روی مدل‌های صنعتی.

۲.۴ ورودی‌های ارزیابی

جدول ۱.۴ چهار ورودی و مجموعه مشاهده‌های آن‌ها را نشان می‌دهد. نمونه tiny یک سامانه کوچک و قابل بررسی دستی است که دو مسیر با مدت کل برابر دارد. سه نمونه دیگر از نمودارهای مطالعه موردی خانه هوشمند استخراج شده‌اند. در هر سه، تنها پیام‌های activateh، getSense و switchoff قابل مشاهده‌اند و همه پیام‌های دیگر به τ تبدیل می‌شوند.

جدول ۱.۴: ورودی‌ها و هدف هر مورد ارزیابی

مورد	تعامل‌های قابل مشاهده	هدف
tiny	poll	بررسی دستی ادغام دو مسیر با مدت کل یکسان.
smart-home	getSense, activateh, switchoff	نمونه پایه خانه هوشمند با یک تأخیر ده‌واحدی.
smart-home-tc2step	همان سه تعامل	بیان همان دوره با تأخیرهای هفت و سه‌واحدی و یک گام داخلی میان آن‌ها.
smart-home-notify	همان سه تعامل	نمونه بزرگ‌تر با مؤلفه اعلان و تقسیم دوره به سه و هفت واحد.

سه فایل خانه هوشمند از روی نمودارهای ارائه‌شده برای پروژه به جدول یال تبدیل شده‌اند. شمار حالت‌ها و یال‌ها، جهت یال‌ها، حالت آغازین و قواعد یکسان‌سازی نام‌ها در مستند منشأ داده ثبت شده است. فایل سازگار با قالب افرا نیز از روی نمایش Graphviz رسمی بازسازی شده، نه اینکه مستقیماً از اجرای افرا کپی شده باشد. این تفاوت بر ساختار یال‌های مورد ارزیابی اثر شناخته‌شده‌ای ندارد، اما منشأ بازسازی شده داده دامنۀ اعتبار خارجی نتیجه را محدود می‌کند.

۳.۴ شرایط اجرا و سنجه‌ها

نتایج ثبت‌شده با نسخه 1.0.0 ابزار و کامیت 3d63904 تولید شده‌اند. محیط اجرا Darwin 23.6.0 روی معماری arm64 با ۸ پردازنده منطقی و OpenJDK 17.0.10 بوده است. اسکرپت ارزیابی هر ورودی را در

یک فرایند جدا اجرا کرده و فایل metrics.json همان اجرا را در کنار خروجی‌های ساختاری ذخیره کرده است [۱۵].

اگر $|S_o|$ و $|E_o|$ به ترتیب تعداد حالت‌ها و گذارهای ورودی و $|S_r|$ و $|E_r|$ مقادیر مدل کاهش یافته باشند، نسبت‌های گزارش شده از رابطه‌های زیر به دست می‌آیند:

$$\rho_S = 1 - \frac{|S_r|}{|S_o|}, \quad \rho_E = 1 - \frac{|E_r|}{|E_o|}.$$

ستون «پس از پالایش زمانی» اندازه نمایش میانی بعد از تبدیل تأخیرهای چندواحدی به یال‌های واحد است و نباید با اندازه ورودی اشتباه گرفته شود. «تعداد رده‌ها» نیز روی همین نمایش پالایش شده محاسبه می‌شود. همان‌طور که در فصل قبل توضیح داده شد، برخی رده‌های صرفاً زمانی هنگام ساخت خروجی روی یک یال فشرده می‌شوند؛ ازاین‌رو، تعداد رده‌ها ممکن است از تعداد حالت‌های مدل نهایی بیشتر باشد.

۴.۴ نتایج کاهش

جدول ۲.۴ اندازه‌های اصلی را نشان می‌دهد. واریسی‌گر مستقل، رابطه مدل ورودی و مدل کاهش یافته را برای هر چهار مورد پذیرفته است. این پذیرش به معنای مینیمال بودن مدل خروجی نیست؛ نتیجه پشتیبانی شده این است که خروجی، مطابق معنای زمان گسسته انتخاب شده، با ورودی شبیه‌سازی دوسویه ضعیف زمان‌دار دارد.

جدول ۲.۴: شمار حالت‌ها، گذارها و نتیجه کاهش

مورد	ورودی	پس از پالایش زمانی	رده‌ها	خروجی
tiny	۶ حالت و ۷ گذار	11 / 12	۵	2 / 2
smart-home	16 / 18	34 / 36	۲۸	10 / 12
smart-home-tc2step	25 / 28	41 / 44	۲۸	10 / 12
smart-home-notify	42 / 52	85 / 95	۲۸	10 / 12

در نمونه کوچک، شش حالت و هفت گذار به دو حالت و دو گذار کاهش یافته‌اند. در سه نمونه خانه هوشمند، با وجود تفاوت اندازه ورودی‌ها، پالایش افراز در هر مورد ۲۸ رده ساخته و مرحله فشرده‌سازی زمانی به مدل‌هایی

جدول ۳.۴: نسبت کاهش حالت‌ها و گذارها

مورد	کاهش حالت‌ها	کاهش گذارها
tiny	66.7 درصد	71.4 درصد
smart-home	37.5 درصد	33.3 درصد
smart-home-tc2step	60.0 درصد	57.1 درصد
smart-home-notify	76.2 درصد	76.9 درصد

با ۱۰ حالت و ۱۲ گذار رسیده است. بیشترین نسبت کاهش میان ورودی‌های موجود به smart-home-notify تعلق دارد: 76.2 درصد از حالت‌ها و 76.9 درصد از گذارها حذف شده‌اند. این عدد فقط برای مجموعه مشاهده سه‌عضوی جدول ۱.۴ معتبر است؛ انتخاب مشاهده‌های بیشتر یا کمتر می‌تواند افراز دیگری بسازد.

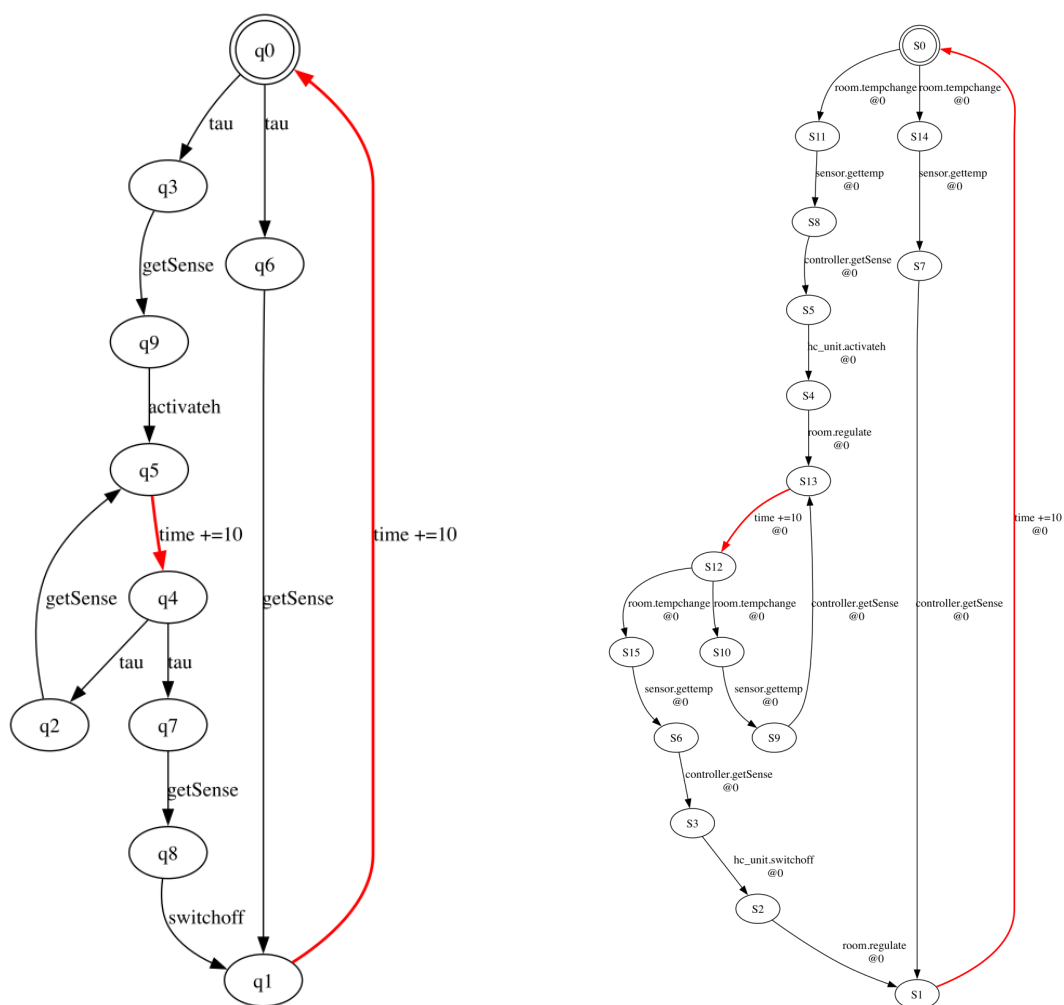
۵.۴ مطالعه موردی خانه هوشمند

شکل ۱.۴ فضای حالت پایه و خارج قسمت آن را کنار هم قرار می‌دهد. یال‌های قرمز گذر زمان را نشان می‌دهند. نمودار نخست ۱۶ حالت و ۱۸ گذار دارد و نمودار دوم همان رفتار قابل مشاهده را با ۱۰ حالت و ۱۲ گذار نمایش می‌دهد. هر دو تصویر با ابزارهای همان مخزن از ورودی smart-home.statespace تولید شده‌اند [۱۵].

سه ورودی خانه هوشمند یک رفتار را با جزئیات داخلی متفاوت بیان می‌کنند. بررسی بازنمایی‌های اصلی خروجی نشان می‌دهد هر سه، صرف‌نظر از نام حالت‌ها، به ساختار خارج‌قسمتی یکسان رسیده‌اند. این نتیجه با هم‌ارزی جفت‌به‌جفت آن‌ها سازگار است. افزون بر پاسخ مثبت دستور equivalent، این ساختار مشترک شاهدهای برای همگرایی سه ورودی فراهم می‌کند.

با این حال، این مشاهده قضیه‌ای دربارهٔ یکتا بودن خارج‌قسمت برای همهٔ سامانه‌های هم‌ارز نیست؛ هم‌ارزی ضعیف در حالت کلی لزوماً خارج‌قسمت کانونی تولید نمی‌کند.

تفاوت ۲۸ ردهٔ جدول ۲.۴ با ۱۰ حالت خروجی ناشی از حذف رده‌های هم‌ارز نیست. پالایش واحدی، لحظه‌های میانی گذر زمان را به حالت تبدیل می‌کند و افراز نیز برای آن‌ها رده می‌سازد. مرحلهٔ فشرده‌سازی، زنجیره‌های بدون انتخاب را دوباره به یال‌های چندواحدی تبدیل می‌کند. نگاشت این رده‌ها در partition.json باقی می‌ماند؛ بنابراین، هر حالت ورودی یا به یک حالت خروجی و یا به نقطه‌ای مشخص روی یک یال زمان‌دار



(ب) مدل خارج‌قسمتی واری شده

(الف) فضای حالت ورودی

شکل ۱.۴: فضای حالت خانه هوشمند پیش و پس از کاهش ضعیف زمان‌دار.

قابل ردیابی است.

۶.۴ مطالعه موردی مدل‌های ربکا

مدل‌های خانه هوشمند به صورت جدول یال و از روی نمودارهای ارائه شده بازسازی شده‌اند. برای آنکه ارزیابی به ورودی‌هایی هم تکیه کند که تمام مسیر تولید را از کد ربکا تا فایل فضای حالت طی کرده‌اند، هفت مدل ربکای زمان‌دار نوشته و با همان زنجیره ابزار رسمی به فضای حالت تبدیل شدند. این هفت مدل سه خانواده می‌سازند و اعضای هر خانواده قرار است رفتار قابل مشاهده یکسانی داشته باشند، اما آن رفتار را با ساختار داخلی و تقسیم‌بندی متفاوتی از زمان بیان کنند.

مسیر تولید بدون استفاده از رابط گرافیکی افرا و به صورت زیر است: کامپایلر ربکا و RMC از مدل ربکا یک واریسی‌گر مدل به زبان C++ می‌سازند، آن برنامه کامپایل و اجرا می‌شود، و خروجی صادر شده آن همان فایل statespace است که ورودی ابزار این پروژه است [۲۱، ۱۵]. نمودارهای فضای حالت ورودی نیز با همان StateSpaceTransformer رسمی افرا به Graphviz تبدیل شده‌اند، نه با نمایش دهنده این پروژه؛ بنابراین آنچه در شکل‌های این بخش دیده می‌شود، همان چیزی است که افرا نشان می‌دهد. در نمادگذاری افرا، یال قرمز و ضخیم گذر زمان را نشان می‌دهد و یال سیاه اجرای یک پیام خدمت است؛ عدد پس از نشان @ زمان مطلق اجرا و نشان shift بازنشانی ساعت هنگام آغاز دوباره یک رفتار تکراری است.

نتایج این بخش با نسخه 1.0.0 ابزار و کامیت 9adbb65 و در همان محیط اجرای بخش پیشین تولید شده‌اند. متن کامل هر هفت مدل ربکا در پیوست آمده است.

۱.۶.۴ خانواده mood: تأخیر یکپارچه در برابر تأخیر شکسته

این خانواده دقیقاً همان وضعیتی را می‌سازد که بخش ۶.۳ به صورت انتزاعی توضیح داد. برنامه‌های ۱.۴ و ۲.۴ دو مدل این خانواده‌اند. در هر دو، تنها پیام‌های mood.LAUGH و mood.CRY قابل مشاهده‌اند و بقیه پیام‌ها به τ تبدیل می‌شوند.

```
// observable: mood.LAUGH, mood.CRY
reactiveclass MoodSystem(2) {
```

1

2

statevars {	3
boolean happyBranch;	4
}	5
MoodSystem() {	6
happyBranch = false;	7
self.start();	8
}	9
msgsrv start() {	10
happyBranch = ?(true, false);	11
if (happyBranch) {	12
self.happiness() after(2);	13
} else {	14
self.sadness() after(3);	15
}	16
}	17
msgsrv happiness() {	18
self.laugh() after(3);	19
}	20
msgsrv laugh() {	21
self.start();	22
}	23
msgsrv sadness() {	24
self.cry() after(5);	25
}	26
msgsrv cry() {	27
self.start();	28
}	29
}	30
main {	31
MoodSystem mood():();	32
}	33

برنامه: 4.1: mood1

// observable: mood.LAUGH, mood.CRY	1
reactiveclass MoodSystem(2) {	2
statevars {	3
boolean laughBranch;	4
}	5
MoodSystem() {	6
self.start();	7
}	8
msgsrv start() {	9
laughBranch = ?(true, false);	10
if (laughBranch) {	11
self.laugh() after(5);	12
} else {	13
self.cry() after(8);	14
}	15

}	16
msgsrv laugh() {	17
self.start();	18
}	19
msgsrv cry() {	20
self.start();	21
}	22
}	23
main {	24
MoodSystem mood():();	25
}	26

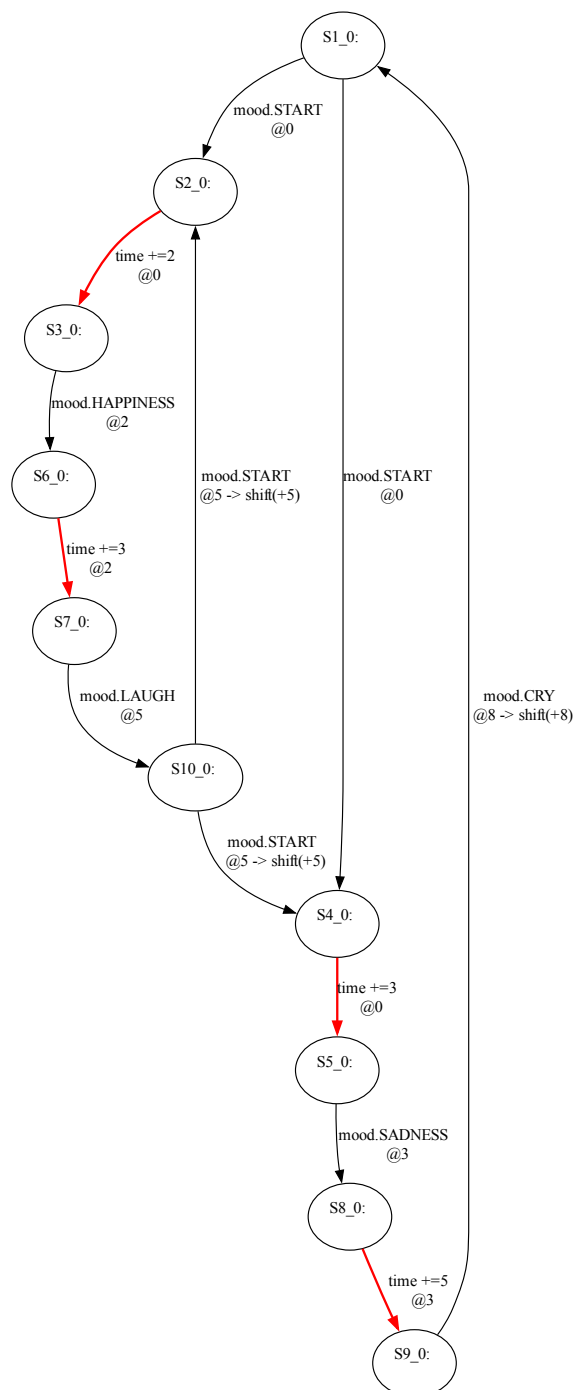
برنامه ۴.۲: mood2

تفاوت دو مدل در این است که mood1 میان آغاز دوره و خنده یک پیام میانی happiness دارد و انتظار را به ۲ و ۳ واحد می‌شکند، درحالی‌که mood2 همان دوره را با یک انتظار پنج‌واحدی بیان می‌کند. چون happiness قابل مشاهده نیست، ناظر بیرونی در هر دو مدل فقط یک چیز می‌بیند: خنده در لحظه پنج. شاخه اندوه نیز به همین ترتیب، در mood1 با $5 + \tau + 3$ و در mood2 با یک انتظار هشت‌واحدی بیان شده است.

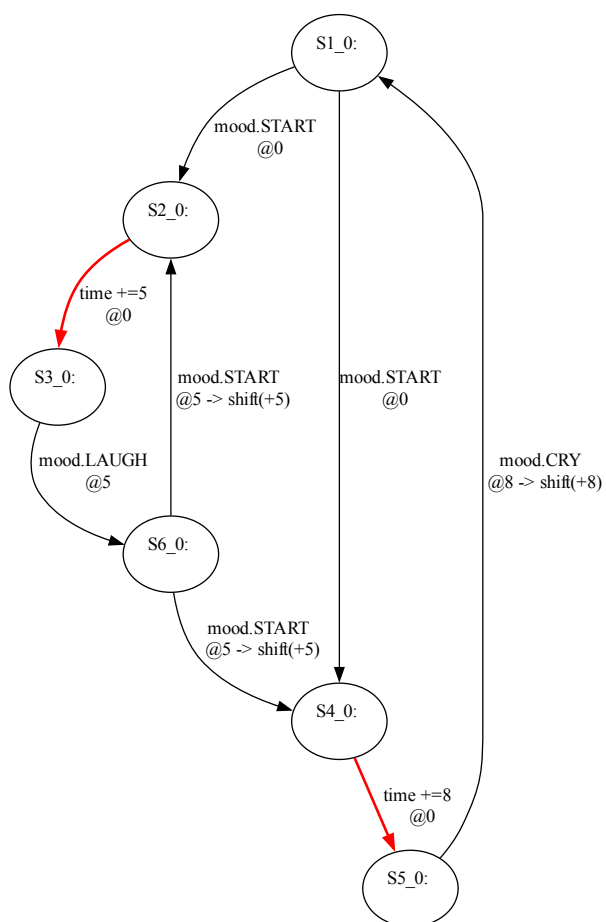
شکل‌های ۲.۴ و ۳.۴ فضای حالت تولیدشده برای این دو مدل را نشان می‌دهند. در شکل ۲.۴ زنجیره بالای نمودار دقیقاً همان چیزی است که بخش ۶.۳ توصیف کرد: یک یال تأخیر دو واحدی، سپس یک گام داخلی، سپس یک یال تأخیر سه واحدی. در شکل ۳.۴ همان مسیر با یک یال تأخیر پنج‌واحدی ظاهر شده است.

دستور equivalent این دو مدل را در معنای پیش‌فرض unit هم‌ارز اعلام می‌کند. اگر همان مقایسه با گزینه strict تکرار شود، یعنی یال‌های تأخیر شکسته نشوند، پاسخ منفی می‌شود. این دقیقاً همان شکستی است که بخش ۶.۳ پیش‌بینی کرد: حالت میانی «دو واحد گذشته است» در mood1 صریحاً وجود دارد، اما در mood2 تا وقتی یال پنج‌واحدی شکسته نشود چنین حالتی برای جفت‌شدن با آن در دسترس نیست. بنابراین این جفت، نمونه‌ای واقعی و تولیدشده از ربکا است که ضرورت شکستن یال‌های تأخیر را نشان می‌دهد.

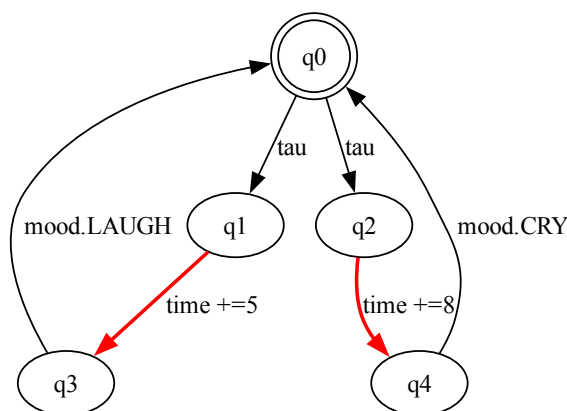
کاهش هر دو مدل به مدل خارج‌قسمتی یکسانی می‌رسد که شکل ۴.۴ آن را نشان می‌دهد: پنج حالت و شش گذار، با دو شاخه داخلی که یکی به انتظار پنج‌واحدی و خنده و دیگری به انتظار هشت‌واحدی و گریه می‌رسد. همه پیام‌های میانی از مدل خروجی حذف شده‌اند و آنچه مانده است، تنها کنش‌های قابل مشاهده و زمان‌بندی آن‌هاست.



شکل ۲.۴: فضای حالت تولید شده برای mood1؛ ۱۰ حالت و ۱۲ گذار. نمودار با StateSpaceTransformer افرا تولید شده است.



شکل ۳.۴: فضای حالت تولیدشده برای mood2؛ ۶ حالت و ۸ گذار.



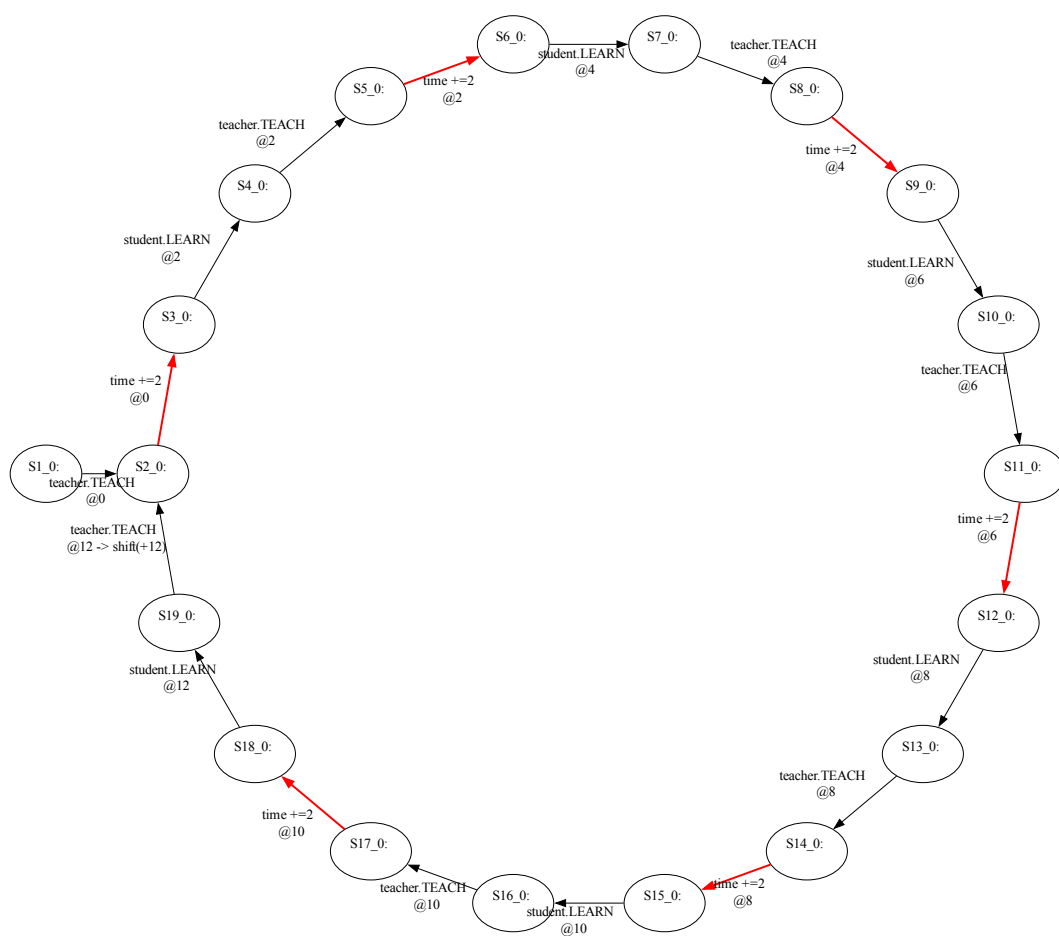
شکل ۴.۴: مدل خارج‌قسمتی مشترک mood1 و mood2؛ ۵ حالت و ۶ گذار.

۲.۶.۴ خانواده teacher: متغیر حالتی که رفتار قابل مشاهده را تغییر نمی‌دهد

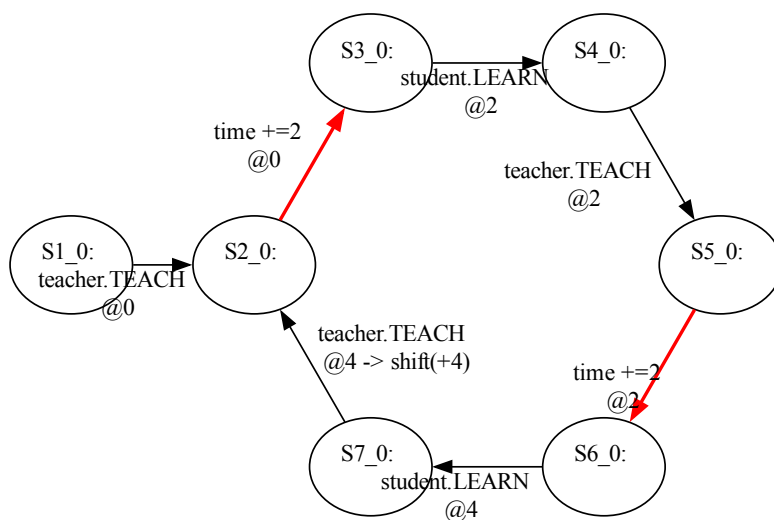
سه مدل این خانواده یک چرخه ساده را بیان می‌کنند: معلم درس می‌دهد، دو واحد زمان می‌گذرد، دانش‌آموز یاد می‌گیرد و دوباره به معلم پیام می‌دهد. تنها پیام قابل مشاهده teacher.TEACH است. تفاوت سه مدل در یک شمارنده پیمانه‌ای سه‌تایی به نام wand است که در teacher1 در کلاس معلم به‌روز می‌شود، در teacher3 در کلاس دانش‌آموز، و در teacher2 اصلاً به‌روز نمی‌شود. این شمارنده در هیچ شرطی به کار نمی‌رود و هیچ پیامی را تغییر نمی‌دهد؛ فقط فضای حالت را بزرگ‌تر می‌کند.

اثر آن در اندازه فضای حالت آشکار است: teacher3 و teacher1 هر دو ۱۹ حالت و ۱۹ گذار دارند، درحالی‌که teacher2 با ۷ حالت و ۷ گذار همان رفتار را بیان می‌کند. فضای حالت تولیدشده برای teacher1 و teacher3 تا حد نام حالت‌ها یکسان است؛ ازاین‌رو شکل ۵.۴ هر دو را نشان می‌دهد.

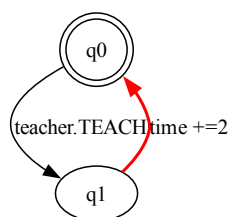
هر سه مدل جفت‌به‌جفت هم‌ارزند و هر سه به مدل خارج‌قسمتی یکسان شکل ۷.۴ می‌رسند: دو حالت و دو گذار، یعنی همان چرخه «درس‌دادن، سپس دو واحد انتظار». این نمونه نشان می‌دهد که کاهش، درست همان جزئیاتی را حذف می‌کند که برای مجموعه مشاهده انتخاب‌شده بی‌اثرند.



شکل ۵.۴: فضای حالت تولیدشده برای teacher1 و teacher3؛ ۱۹ حالت و ۱۹ گذار.



شکل ۶.۴: فضای حالت تولیدشده برای teacher2؛ ۷ حالت و ۷ گذار.



شکل ۷.۴: مدل خارج‌قسمتی مشترک هر سه مدل teacher؛ ۲ حالت و ۲ گذار.

۳.۶.۴ خانواده chain: زنجیره پیام در برابر بیان فشرده

این خانواده دو مدل دارد که یک فرایند تصمیم‌گیری را بیان می‌کنند. در chain1، کنترل‌کننده و کارگر پیام‌های میانی prepare، forwardDecision، complete و restart را رد و بدل می‌کنند و هر یک از این گام‌ها یک واحد زمان می‌برد. در chain2، همان فرایند بدون پیام‌های میانی و با تأخیرهای یکجا بیان شده است. تنها پیام‌های worker.DECIDE، controller.APPROVED و controller.REJECTED قابل مشاهده‌اند.

برخلاف خانواده teacher، در اینجا تفاوت فقط اندازه فضای حالت نیست؛ ساختار زمانی نیز متفاوت بیان شده است. chain1 با ۲۳ حالت و ۲۵ گذار و chain2 با ۱۰ حالت و ۱۲ گذار تولید می‌شوند و شکل‌های ۸.۴ و ۹.۴ آن‌ها را نشان می‌دهند. با این حال، ابزار این دورا هم‌ارز اعلام می‌کند و هر دو به مدل خارج‌قسمتی یکسان شکل ۱۰.۴ با هفت حالت و هشت گذار می‌رسند؛ مدلی که در آن دو شاخه تأیید و رد با مدت‌های سه و شش واحد از هم جدا می‌شوند.

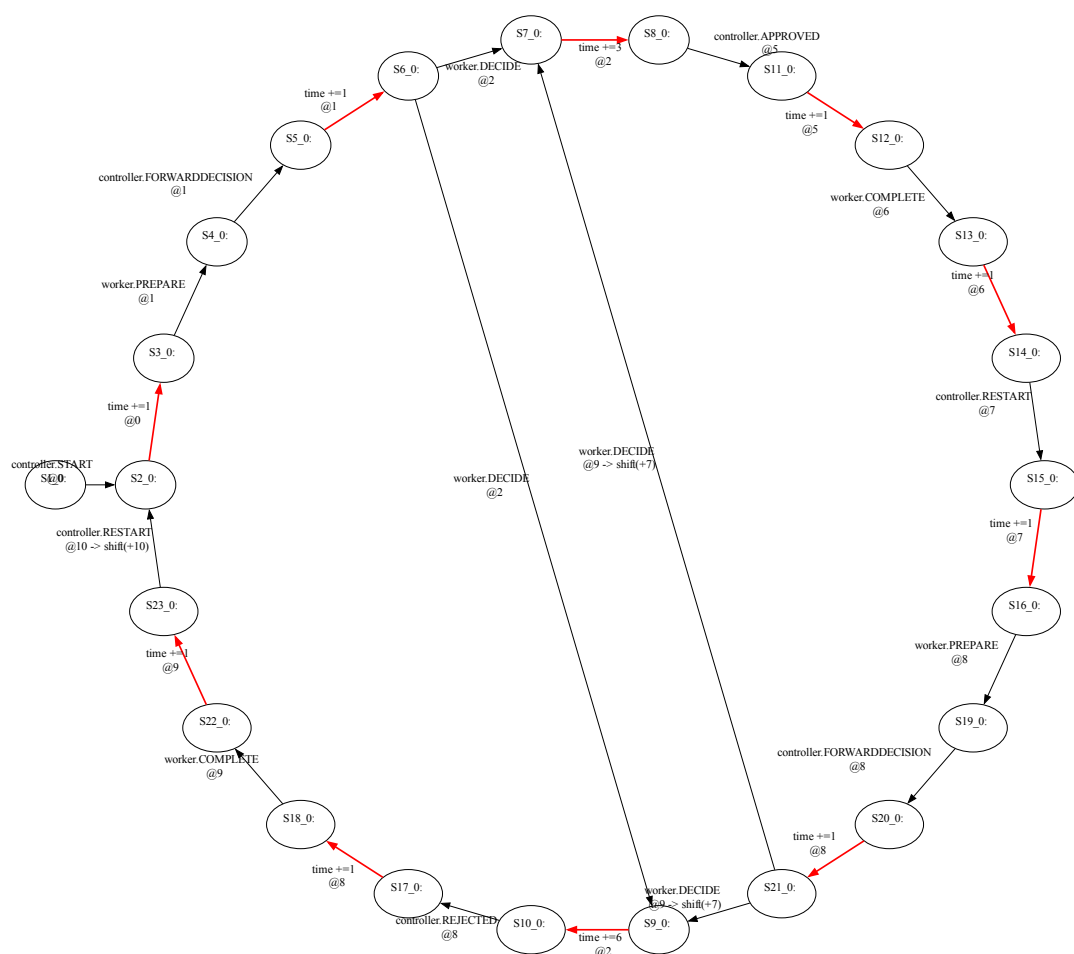
۴.۶.۴ جمع‌بندی عددی

جدول ۴.۴ اندازه‌ها و نسبت کاهش هر هفت مدل را گرد هم می‌آورد. واری‌گر مستقل رابطه ورودی و خروجی را برای هر هفت مورد پذیرفته است.

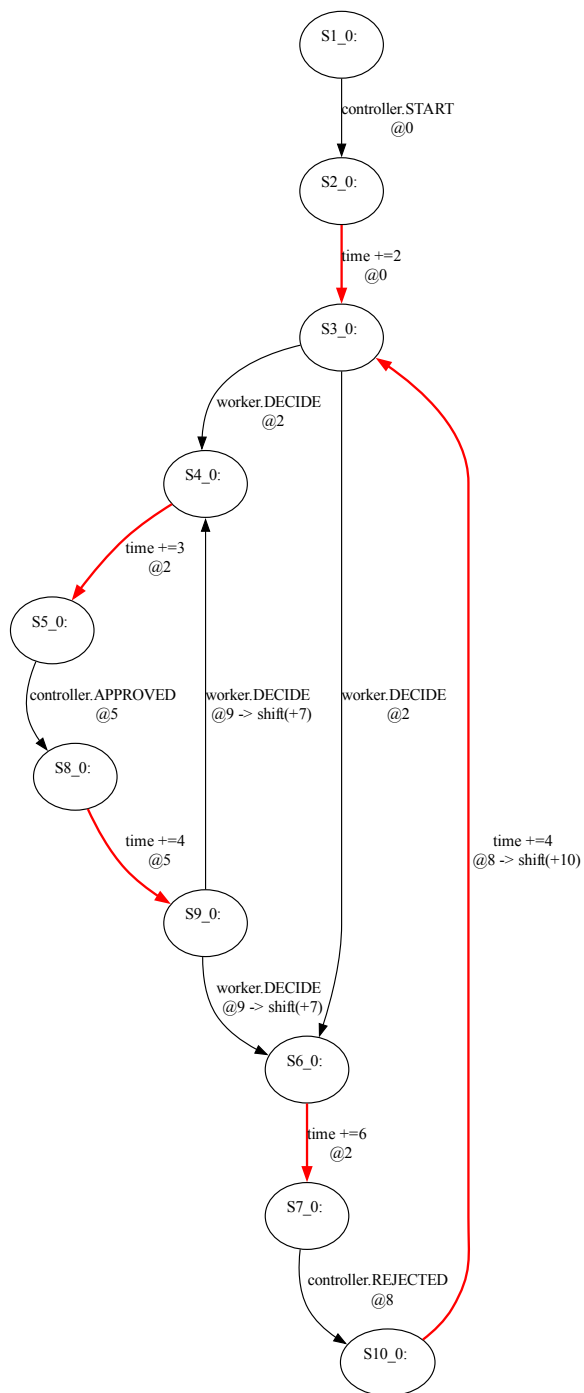
جدول ۴.۴: کاهش فضای حالت‌های تولیدشده از مدل‌های ربکا

مدل	ورودی (حالت/گذار)	خروجی (حالت/گذار)	کاهش حالت	کاهش گذار
mood1	10 / 12	5 / 6	٪50.0	٪50.0
mood2	6 / 8	5 / 6	٪16.7	٪25.0
teacher1	19 / 19	2 / 2	٪89.5	٪89.5
teacher2	7 / 7	2 / 2	٪71.4	٪71.4
teacher3	19 / 19	2 / 2	٪89.5	٪89.5
chain1	23 / 25	7 / 8	٪69.6	٪68.0
chain2	10 / 12	7 / 8	٪30.0	٪33.3

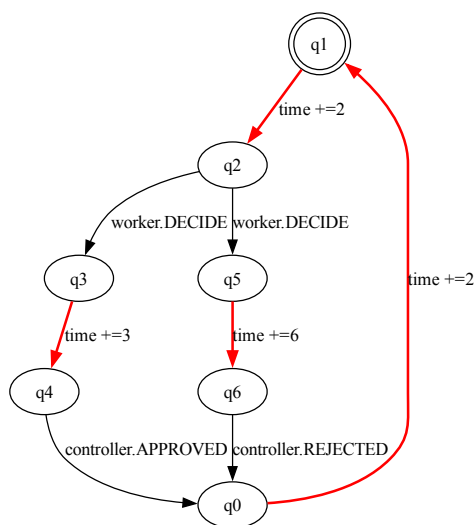
دو مشاهده از این جدول قابل استخراج است. نخست اینکه نسبت کاهش به‌تنهایی معنا ندارد و باید در کنار مجموعه مشاهده‌های همان اجرا خوانده شود؛ عدد 89.5 درصد teacher1 فقط برای مجموعه‌ای معتبر است که تنها عضو آن teacher.TEACH باشد. دوم اینکه اعضای یک خانواده با وجود اندازه‌های ورودی بسیار متفاوت،



شکل ۸.۴: فضای حالت تولیدشده برای chain1؛ ۲۳ حالت و ۲۵ گذار.



شکل ۹.۴: فضای حالت تولیدشده برای chain2؛ ۱۰ حالت و ۱۲ گذار.



شکل ۱۰.۴: مدل خارج‌قسمتی مشترک chain1 و chain2؛ ۷ حالت و ۸ گذار.

به مدل خروجی یکسانی می‌رسند؛ همان چیزی که از یک کاهش مبتنی بر هم‌ارزی انتظار می‌رود. این مشاهده قضیه‌ای دربارهٔ یکتا بودن مدل خارج‌قسمتی برای همهٔ سامانه‌های هم‌ارز نیست.

۷.۴ حساسیت به معنای زمان و مجموعهٔ مشاهده‌ها

جدول ۵.۴ نتیجهٔ مقایسهٔ نمونهٔ پایه با نسخهٔ tc2step را تحت دو تفسیر بخش ۶.۳ نشان می‌دهد. با جمع‌پذیری زمان، تأخیر ده‌واحدی می‌تواند پاسخ اجرای $3 + \tau + 7$ باشد و دو حالت آغازین در یک رده قرار می‌گیرند. در حالت strict، هر طرف باید یک یال منفرد با همان مدت داشته باشد و مقایسه رد می‌شود.

جدول ۵.۴: حساسیت نتیجه به تفسیر تأخیر

حالت	ملاک تطبیق تأخیر	نتیجه
unit	اجرای بدون کنش قابل مشاهده با مدت کل برابر	هم‌ارز
strict	یک یال منفرد با برچسب مدت برابر	ناهم‌ارز

این آزمون نشان می‌دهد انتخاب معنای زمان فقط یک تفاوت پیاده‌سازی نیست و مستقیماً حکم هم‌ارزی را تغییر می‌دهد. وجود هر دو حالت در رابط برنامه امکان بازبینی فرض را حفظ می‌کند. نتیجهٔ مثبت اصلی فصل

بر حالت unit تکیه دارد و نباید به خوانش strict تعمیم داده شود.

آزمون انتهابه‌انتها همچنین بررسی می‌کند تغییر مجموعه مشاهده‌ها افزاز را تغییر می‌دهد و همه اجراها همچنان خارج‌قسمتی پذیرفته‌شده تولید می‌کنند. این نتیجه مورد انتظار است: پنهان کردن تعامل‌های بیشتر، قدرت تمایز ناظر را کاهش می‌دهد و ممکن است حالت‌های بیشتری قابل ادغام شوند. در مقابل، یکسان‌سازی خودکار بزرگی و کوچکی حروف انجام نمی‌شود؛ مقایسه نام GETSENSE در یک خروجی با getSense در ورودی دیگر نتیجه منفی می‌دهد.

۸.۴ نتایج واری و رابطه خروجی

جدول ۶.۴ تعداد زوج‌های رابطه و تعهدهای انتقال بررسی شده را از فایل‌های metrics.json گزارش می‌کند. منظور از تعهد انتقال، یک حرکت ضعیف در یکی از دو جهت است که باید پاسخ متناظر آن پیدا شود. در همه موارد، فهرست نقض‌ها تهی بوده است.

جدول ۶.۴: خلاصه واری واری مستقل مدل‌های کاهش یافته

مورد	زوج‌های مرتبط	تعهدهای بررسی شده	نتیجه
tiny	۱۱	۵۲	پذیرفته
smart-home	۳۴	۱۷۲	پذیرفته
smart-home-tc2step	۴۱	۲۱۴	پذیرفته
smart-home-notify	۸۵	۴۵۴	پذیرفته

صرف پذیرفته‌شدن خروجی صحیح نشان نمی‌دهد واری‌گر توان کشف خطا دارد. برای کنترل این مسئله، پنج جهش روی خارج‌قسمت اعمال شده است: حذف یال، تغییر برجسب تعامل، تغییر مدت، تغییر مقصد و افزودن یال. هر پنج مدل جهش‌یافته رد شده‌اند. یک آزمون جدا نیز با تغییر مدت در ورودی، حکم هم‌ارزی مثبت را به منفی تبدیل می‌کند. این نتیجه‌ها احتمال پذیرش همیشگی یا بررسی ناقص یک‌طرفه را کاهش می‌دهند، هرچند جای اثبات صوری خود برنامه را نمی‌گیرند.

۹.۴ پوشش آزمون‌ها

ساخت ثبت‌شده شامل ۱۱۶ آزمون مبتنی بر JUnit است که بدون شکست، خطا یا صرف‌نظر اجرا شده‌اند. توزیع آن‌ها در جدول ۷.۴ آمده است. افزون بر آن، اسکریپت انتهابه‌انتها ۶۲ بررسی را از طریق فایل JAR بسته‌بندی‌شده اجرا کرده و همه آن‌ها پذیرفته شده‌اند.

جدول ۷.۴: تعداد آزمون‌ها بر حسب هدف

تعداد	گروه آزمون
۲۱	خط مبنای قدیمی در قالب آزمون جعبه‌سیاه
۱۲	نمونه‌های پذیرش خائن هوشمند و جهش‌های آن‌ها
۲۴	معنای گذار ضعیف، بستارها، تأخیرها و پایداری افزاز
۲۳	ساختار خارج‌قسمت، پذیرش رابطه و جهش‌های واریسی‌گر
۱۷	خواننده افرا و ورودی‌های نامعتبر
۸	مرز معماری و قرارداد آداپتور
۵	فضای حالت‌های تولیدشده از مدل‌های ربکا
۳	یکتایی شناسه حالت‌های ساخته‌شده
۳	تولید نمایش Graphviz
۱۱۶	جمع آزمون‌های JUnit
۶۲	بررسی‌های انتهابه‌انتها

آزمون انتهابه‌انتها وجود و محتوای پنج مصنوع خروجی، کدهای خروج، ورودی نامعتبر و اثر مجموعه مشاهده‌ها را بررسی می‌کند. حساسیت معنای زمان، جهش یک مدل و قطعی‌بودن فایل‌های ساختاری نیز در همین سطح آزموده می‌شوند.

این آزمون فقط برنامه بسته‌بندی‌شده را از بیرون اجرا می‌کند. ازاین‌رو، موفقیت آن شواهدی جدا از فراخوانی مستقیم کلاس‌های داخلی فراهم می‌آورد [۱۵].

۱۰.۴ زمان اجرا و حافظه

جدول ۸.۴ زمان درون‌فرایندی گزارش شده توسط ابزار، زمان دیواری اندازه‌گیری شده توسط اسکریپت و بیشینه حافظه پشته جاوا را نشان می‌دهد. حافظه با واحد مگابایت دهدهی گزارش شده است.

جدول ۸.۴: زمان و بیشینه حافظه در اجرای ثبت شده

مورد	زمان درون‌فرایندی (میلی ثانیه)	زمان دیواری (میلی ثانیه)	حافظه (مگابایت)
tiny	۵۰	130.3	6.2
smart-home	۵۵	137.0	7.2
smart-home-tc2step	۵۸	139.5	7.2
smart-home-notify	۶۳	153.3	8.3

فاصله زمان درون‌فرایندی و زمان دیواری نشان می‌دهد هزینه راه‌اندازی ماشین مجازی جاوا سهم قابل توجهی از اجرای این نمونه‌های کوچک را تشکیل می‌دهد. اختلاف زمان‌ها کم است، اما چهار نقطه و یک اجرا برای هر مدل برای برآزش روند یا مقایسه‌ی مجانبی کافی نیست. نتیجه محتاطانه این است که همه نمونه‌های موجود با حافظه کمتر از ۹ مگابایت و زمان دیواری کمتر از ۱۶۰ میلی ثانیه در محیط ثبت شده پردازش شده‌اند؛ این مشاهده به ورودی‌های بزرگ‌تر قابل تعمیم نیست.

۱۱.۴ جمع‌بندی

هر چهار ورودی سنجه‌ای و هر هفت فضای حالت تولید شده از ربکا به مدل کوچک‌تری رسیده‌اند و واریسی‌گر مستقل رابطه خروجی همه آن‌ها را پذیرفته است. سه بیان متفاوت از خانه هوشمند، با مجموعه مشاهده یکسان و معنای جمع‌پذیر زمان، به خارج قسمت‌هایی هم‌ساختار با ۱۰ حالت و ۱۲ گذار رسیده‌اند. آزمون حالت strict نشان داد این نتیجه به تصمیم معنایی فصل ۳ وابسته است.

مجموعه آزمون‌ها، جهش‌های منفی و مصنوعات قابل ردیابی از نتایج موجود پشتیبانی می‌کنند. با این حال، اندازه کوچک ورودی‌ها و منشأ بازسازی شده فایل اجرا نمی‌دهد درباره مقیاس‌پذیری یا پوشش کامل زنجیره رسمی اجرا ادعایی مطرح شود.

فصل ۵

بحث و نتیجه‌گیری

۱.۵ مرور مسیر پروژه

این پروژه از یک مسئله عملی در زنجیره تحلیل ربکا آغاز شد: فضای حالت زمان‌دار تولیدشده توسط افرا می‌تواند جزئیاتی داشته باشد که برای سطح مشاهده انتخابی کاربر ضروری نیستند، اما تحلیل‌گر همچنان باید همه آن‌ها را نگهداری و پیمایش کند. تعریف شبیه‌سازی دوسویه ضعیف زمان‌دار مبنایی برای نادیده گرفتن کنش‌های داخلی با حفظ رفتار قابل مشاهده و زمان فراهم می‌کند [۲۶، ۱۲]. فاصله میان این تعریف و یک ابزار قابل استفاده شامل چند مسئله مهندسی و معنایی بود: قرارداد واقعی فایل افرا، هویت تعامل‌ها، تعبیر تأخیرهای چندمرحله‌ای، ساخت مدل خارج‌قسمتی و واریسی نتیجه تحویلی.

کار اصلی پروژه پیاده‌سازی این فرایند در ابزار awtr بود. ابزار فایل statespace را می‌خواند، تعامل‌های خارج از مجموعه مشاهده را پنهان می‌کند، بخش دسترس‌پذیر مدل را نگه می‌دارد، تأخیرها را مطابق معنای انتخابی پالایش می‌کند و پس از ساخت رابطه گذار ضعیف، افراز پایدار و مدل خارج‌قسمتی را به دست می‌آورد. واریسی جدایه رابطه مدل پالایش‌شده و خروجی را در هر دو جهت بررسی می‌کند. نتیجه همراه با نداشت افراز، منشأ یال‌ها و سنجه‌های اجرا ذخیره می‌شود تا صرفاً به یک پاسخ مثبت بدون شاهد اکتفا نشود.

۲.۵ پاسخ به پرسش‌های پروژه

۱.۲.۵ خواندن و مستقل کردن خروجی افرا

پرسش نخست درباره امکان مصرف مستقیم خروجی زمان‌دار افرا بود. بررسی تولیدکننده و خواننده‌های رسمی نشان داد فایل واقعی TTS قراردادی دارد که تنها از روی طرح داده روشن نمی‌شود؛ از جمله، عنصر ریشه ممکن است در انتهای فایل بسته نشده باشد و نخستین حالت نقش حالت آغازین را داشته باشد. خواننده پیاده‌سازی شده این موارد را بدون تغییر دستی فایل مدیریت و داده را پشت رابطی مستقل از افرا به مدل دامنه تبدیل می‌کند. آزمون قرارداد نشان می‌دهد منبع فایل و منبع درون‌حافظه‌ای به نتیجه یکسان می‌رسند. بنابراین، برای دامنه TTS و فیلدهای مستندشده، پاسخ پرسش نخست مثبت است. این نتیجه به FTTS تعمیم داده نمی‌شود.

۲.۲.۵ کاهش همراه با واریسی مستقل

پرسش دوم ساخت یک مدل کوچک‌تر و واریسی رابطه آن با ورودی بود. هر چهار ورودی سنج‌های و هر هفت فضای حالت تولیدشده از مدل‌های ربکا به خروجی کوچک‌تری رسیدند و واریسی‌گر مستقل همه رابطه‌های تحویلی را پذیرفت. برای سنجش اینکه پذیرش صرفاً نتیجه یک واریسی‌گر همیشه مثبت نیست، پنج نوع تغییر مخرب روی مدل خارج‌قسمتی اعمال شد و واریسی‌گر همه را رد کرد. ۱۱۶ آزمون JUnit و ۶۲ بررسی انتهاه‌انتها نیز بدون شکست اجرا شدند. این شواهد از درستی خروجی‌های موجود پشتیبانی می‌کنند، اما جای اثبات درستی خود پیاده‌سازی یا بازبینی مستقل تعریف را نمی‌گیرند.

واژه «کاهش» در این نتیجه باید دقیق تفسیر شود. مدل خروجی رفتاری را که مجموعه مشاهده و معنای زمان تعیین می‌کند حفظ کرده است؛ با تغییر مجموعه مشاهده، افراز نیز می‌تواند تغییر کند. همچنین، ابزار کوچک‌ترین مدل ممکن یا یک نمایش کانونی را تضمین نمی‌کند. ادعای پشتیبانی شده این است که خارج‌قسمت‌های تولیدشده برای نمونه‌های ارزیابی از ورودی کوچک‌تر و مطابق واریسی‌گر، با آن دارای شبیه‌سازی دوسویه ضعیف زمان‌دار بوده‌اند.

۳.۲.۵ اثر معنای زمان و اندازه کاهش

پرسش سوم به اثر شکستن یال های تأخیر و مقدار کاهش مربوط بود. در حالت پیش فرض unit، هر تأخیر چند واحدی به گام های یک واحدی شکسته می شود؛ بنابراین یک تأخیر ده واحدی می تواند با اجرای $7 + \tau + 3$ یا $7 + \tau + 3$ تطبیق یابد. سه بیان مطالعه موردی خانه هوشمند در این حالت هم ارز شناخته شدند و پس از فشرده سازی به ساختاری با ۱۰ حالت و ۱۲ گذار رسیدند؛ جفت mood1 و mood2 نیز، که از دو مدل واقعی ربکا تولید شده اند، هم ارز شناخته شدند. در حالت strict، که یال ها را دست نخورده نگه می دارد، نه نمونه پایه خانه هوشمند با tc2step هم ارز است و نه mood1 با mood2. این تفاوت، چنان که بخش ۶.۳ توضیح داد، نه یک انتخاب سلیقه ای بلکه نتیجه کنار گذاشتن ویژگی پیوستگی است: بدون شکستن یال ها، الگوریتم روی ساختاری کار می کند که ویژگی های استاندارد یک سامانه انتقال زمان دار را ندارد و آنچه محاسبه می کند دیگر شبیه سازی دوسویه ضعیف زمان دار به معنای تعریف ۳.۸.۲ نیست.

در ورودی کوچک، تعداد حالت ها از ۶ به ۲ و تعداد گذارها از ۷ به ۲ کاهش یافت. برای سه نمونه خانه هوشمند، نسبت کاهش حالت به ترتیب 37.5، 60.0 و 76.2 درصد و نسبت کاهش گذار 33.3، 57.1 و 76.9 درصد بود؛ همه این ارقام با مجموعه مشاهده getSense، activateh و switchoff معنا دارند. در فضای حالت های تولید شده از ربکا، نسبت کاهش حالت میان 16.7 و 89.5 درصد بود و در هر سه خانواده، اعضای هم ارز به مدل خارج قسمتی یکسانی رسیدند. افزایش نسبت کاهش در نمونه هایی که جزئیات داخلی بیشتری دارند قابل انتظار است، اما این تعداد نقطه ارزیابی برای نتیجه گیری درباره روند مقیاس پذیری کافی نیست.

۳.۵ تفسیر تصمیم های طراحی

پالایش تأخیرها به گام های واحد، تعریف جمع پذیر زمان را به مسئله ای متناهی با یک برچسب زمانی واحد تبدیل می کند. مزیت آن سادگی و امکان استفاده از همان پالایش افراز برای کنش و زمان است. هزینه آن افزودن $1 - d$ حالت برای یک یال با مدت d است. فشرده سازی زنجیره های صرفاً زمانی پس از ساخت خارج قسمت، اندازه مدل تحویلی را کاهش می دهد، ولی هزینه نمایش میانی را حذف نمی کند. بنابراین این تصمیم برای ورودی های فعلی مناسب بوده است، نه لزوماً برای مدل هایی با ثابت های زمانی بسیار بزرگ. جداسازی واریسی گر از کاهش دهنده نیز یک مصالحه مهندسی است. واریسی گر بستارها و تعهد های رابطه را

با ساختار داده و مسیر کد متفاوتی محاسبه می‌کند؛ از این رو، بخشی از خطاهای شاخص‌گذاری، ساخت امضا یا فشرده‌سازی می‌تواند آشکار شود. با این حال، هر دو مؤلفه تعبیر معنایی مشترکی دارند و خطای مشترک در همان تعبیر ممکن است از هر دو عبور کند. آزمون جهش اعتماد به توان رد کردن خروجی معیوب را افزایش می‌دهد، اما اثبات صوری واری واری نیست.

خروجی‌های چندگانه نیز دو نیاز متفاوت را پاسخ می‌دهند. `reduced.statespace` برای مشاهده در ابزارهای افرا سودمند است، ولی قالب آن همه منشأها و عضویت رده‌ها را نگه نمی‌دارد. در مقابل، `reduced.json` نتیجه اصلی و بدون اتلاف پروژه است و `partition.json` امکان ردیابی حالت ورودی تا حالت یا یال خروجی را فراهم می‌کند. این تفکیک اجازه می‌دهد سازگاری با ابزار موجود، اطلاعات لازم برای بازتولید نتیجه را محدود نکند.

۴.۵ جایگاه دستاورد

این پایان‌نامه تعریف تازه‌ای از شبیه‌سازی دوسویه یا الگوریتم تازه‌ای برای محاسبه آن ارائه نمی‌کند. تعریف رفتاری از برار و همکاران و ساختار الگوریتم از پالایش افراز گروته و واندرادر گرفته شده است [۵، ۱۰]. سهم پروژه، تبدیل این مبنا به یک مسیر اجرایی و آزموده برای مدل معنایی ربکا است: قرارداد ورودی افرا از کدهای رسمی استخراج شده، نحوه برخورد با تأخیرها از ویژگی پیوستگی سامانه انتقال زمان‌دار استخراج و به صورت گزینه‌ای صریح پیاده‌سازی شده، مدل خارج‌قسمتی همراه با منشأ ساخته شده و نتیجه با پیاده‌سازی جداگانه و آزمون‌های منفی بررسی شده است.

پیش از این، کاربر زنجیره ربکا و افرا ابزاری در اختیار نداشت که فضای حالت زمان‌دار تولیدشده را نسبت به مجموعه‌ای از تعامل‌های قابل مشاهده کاهش دهد؛ وجود چنین ابزاری، مستقل از اندازه کاهشی که روی نمونه‌های خاص به دست می‌آید، خود دستاورد این کار است [۱۸، ۲۱، ۱۵].

از دید کاربردی، ابزار موجود یک نمونه مستقل از خط فرمان است و هنوز جزئی از محیط افرا نیست. در نتیجه، دستاورد فعلی را باید زیرساختی برای ارزیابی روش و ادغام آینده دانست، نه بهبود اثبات‌شده کارایی خود افرا. داده‌های فصل ۴ نشان می‌دهند کاهش روی نمونه‌های موجود رخ داده و هزینه اجرا در محیط ثبت شده کم بوده است؛ اما چون زمان تحلیل بعدی با افرا اندازه‌گیری نشده، نمی‌توان از این نتایج درباره شتاب نهایی واری واری مدل ادعای کمی مطرح کرد.

۵.۵ محدودیت‌ها

محدودیت‌های اصلی نتیجه عبارت‌اند از:

- بزرگ‌ترین ورودی پیش از پالایش زمانی ۴۲ حالت دارد و نماینده یک مدل صنعتی نیست؛
 - ارزیابی چهار ورودی سنج‌های و هفت فضای حالت تولیدشده از سه خانواده مدل ربکا را پوشش می‌دهد و هر زمان اجرا یک‌بار ثبت شده است؛
 - فایل افرا از روی نمایش تولیدشده بازسازی شده و خروجی مستقیم یک اجرای رسمی در دسترس نبوده است؛
 - پشتیبانی زمانی به TTS، زمان گسسته و تأخیرهای صحیح محدود است؛
 - پالایش واحدی می‌تواند برای تأخیرهای بزرگ نمایش میانی بسیار بزرگی بسازد؛
 - نتیجه‌های هم‌ارزی به قرارداد معنایی تأییدشده جمع‌پذیری زمان وابسته‌اند و به خوانش strict تعمیم داده نمی‌شوند؛
 - خارج قسمت واری می‌شود، اما مینیمال یا کانونی بودن آن اثبات نشده است؛
 - سنجش حاضر کاهش فضای حالت را گزارش می‌کند و اثر آن بر زمان یک واریسی مدل کامل را اندازه نمی‌گیرد.
- این محدودیت‌ها نتیجه اصلی روی نمونه‌های ثبت‌شده را نقض نمی‌کنند، اما مرز تعمیم آن را تعیین می‌کنند. به‌ویژه، درصدهای کاهش تنها همراه با مجموعه مشاهده و معنای زمانی همان اجرا قابل تفسیرند.

۶.۵ کارهای آینده

نخستین ادامه طبیعی، افزودن آداپتور درون‌فرایندی و فرمان متناظر در افراسات تا مدل بدون فایل واسط به هسته کاهش تحویل شود. پیش از آن، نمونه بازسازی شده باید با یک خروجی مستقیم و قابل انتشار افرا جایگزین

و قرارداد خواننده روی چند مدل واقعی دیگر بررسی شود. تشخیص صریح FTTS نیز باید افزوده شود تا ابزار به‌جای تفسیر خاموش یک قالب پشتیبانی نشده، خطای روشن ارائه کند.

در سطح معنایی، جمع‌پذیری زمان به‌عنوان قرارداد اصلی روش تثبیت شده است. مقایسه تقاضای با یک پیاده‌سازی مستقل دیگر یا صورت‌بندی مرجع می‌تواند خطر خطای مشترک میان کاهش‌دهنده و واریسی‌گر را کمتر کند. اثبات برخی ناورداهای کلیدی، مانند حفظ رابطه در فشرده‌سازی زنجیره‌زمان، گام بعدی دقیق‌تری از افزودن آزمون‌های مشابه است.

برای مقیاس بزرگ‌تر، می‌توان از پالایش افراز کارآمدتر—برای نمونه صورت $O(m \log n)$ که برای شبیه‌سازی دوسویه انشعابی ارائه شده است [۹]—و نمایش نمادین زمان با ناحیه یا زون استفاده کرد تا نیاز به ساخت یک حالت برای هر واحد تأخیر کاهش یابد. ارزیابی این نسخه باید مجموعه‌ای متنوع از مدل‌های ربکا، تأخیرهای بزرگ‌تر و تعداد اجرای تکراری داشته باشد و علاوه بر اندازه خارج‌قسمت، زمان و حافظه کل جریان واریسی مدل را پیش و پس از کاهش مقایسه کند.

مسیر دیگری نیز وجود دارد که ارزش بررسی دارد و هدف آن رسیدن به همان نتیجه بدون شکستن یال‌های تأخیر است. بخش ۶.۳ نشان داد که اگر یال‌ها دست‌نخورده بمانند، برخی حالت‌ها—مانند حالت میانی q_1 در مثال آن بخش—در طرف مقابل شریکی پیدا نمی‌کنند و ساخت رابطه شکست می‌خورد. به‌جای ساختن آن شریک با شکستن یال، می‌توان تعریف رابطه را اندکی سست کرد و اجازه داد تعدادی حالت بی‌شریک باقی بماند: حالت‌هایی که صرفاً نقطه‌ای میانی روی یک انتظار پیوسته‌اند، هیچ کنش قابل مشاهده‌ای از آن‌ها آغاز نمی‌شود و هیچ انتخاب تازه‌ای نمی‌سازند. چنین حالتی را می‌توان بدون آسیب به رفتار قابل مشاهده از الزام تطبیق معاف کرد و الگوریتم را روی همان گراف ورودی، یعنی در همان حالت strict، اجرا کرد.

کار لازم برای این مسیر، صورت‌بندی دقیق شرطی است که این حالت‌های بی‌شریک را مشخص می‌کند و سپس اثبات اینکه رابطه حاصل هنوز یک شبیه‌سازی دوسویه ضعیف زمان‌دار به معنای تعریف ۳.۸.۲ است. اگر این اثبات به دست آید، نتیجه الگوریتمی است که به همان هم‌ارزی می‌رسد اما هزینه حافظه پالایش واحدی—یعنی افزودن $1 - d$ حالت برای هر یال با مدت d —را نمی‌پردازد. این مسیر مستقل از راه‌حل نمادین بند پیش است و می‌تواند در زمان گسسته نیز به کار رود.

در نهایت، قالب خروجی افرا ظرفیت نگهداری همه منشأهای یک یال خارج‌قسمتی را ندارد. تعریف یک افزونه سازگار برای این فراداده یا نمایش مستقیم آن در رابط افرا می‌تواند قابلیت ردیابی JSON را به جریان تصویری منتقل کند و بررسی دستی علت ادغام حالت‌ها را آسان‌تر سازد.

۷.۵ جمع‌بندی نهایی

نتایج نشان می‌دهند تعریف شبیه‌سازی دوسویه ضعیف زمان‌دار را می‌توان برای خروجی TTS افرا به یک فرایند کاهش قابل اجرا و قابل ردیابی تبدیل کرد. ابزار ساخته‌شده در هر یازده ورودی ارزیابی—چهار ورودی سنج‌ای و هفت فضای حالت تولیدشده از ربکا—مدلی کوچک‌تر تولید کرد، رابطه خروجی‌ها در واریسی مستقل پذیرفته شد و آزمون‌های منفی توانستند تغییرهای معیوب را آشکار کنند. مطالعه موردی همچنین نشان داد یک تصمیم ظاهراً کوچک درباره ترکیب تأخیرها می‌تواند پاسخ هم‌ارزی را تغییر دهد؛ بنابراین، ثبت معنای زمان به‌اندازه خود الگوریتم اهمیت دارد.

این نتیجه یک گام مهندسی برای پیوند مبانی نظری و زنجیره ابزار ربکا و افراست. شواهد موجود برای درستی نمونه‌های بررسی‌شده و امکان کاهش آن‌ها کافی‌اند، اما برای ادعای مقیاس‌پذیری، مینیمال‌بودن خروجی یا بهبود کارایی افرا کافی نیستند. ادامه کار باید بر تثبیت معنای زمان، ورودی‌های واقعی‌تر، ادغام با افرا و ارزیابی در مقیاس بزرگ‌تر متمرکز شود.

پیوست آ

مدل‌های ربکای به‌کاررفته در ارزیابی

این پیوست متن کامل هفت مدل ربکای زمان‌داری را می‌آورد که بخش ۶.۴ از فضای حالت تولیدشده آن‌ها استفاده می‌کند. مدل‌ها همان فایل‌هایی هستند که به کامپایلر ربکا داده شده‌اند؛ تنها تورفتگی و فاصله‌گذاری آن‌ها برای خوانایی مرتب شده است و هیچ دستور یا مقداری تغییر نکرده است. سطر نخست هر فایل، مجموعه تعامل‌های قابل مشاهده آن مدل را به‌صورت توضیح ثبت می‌کند. این توضیح فقط مستندسازی است و به فایل فضای حالت منتقل نمی‌شود؛ مجموعه مشاهده باید هنگام اجرا با گزینه observable- به ابزار داده شود. مدل teacher3 چنین توضیحی ندارد و مجموعه مشاهده خانواده خود، یعنی teacher.TEACH، برای آن به کار رفته است.

۱.آ خانواده mood

متن این دو مدل در برنامه‌های ۱.۴ و ۲.۴ فصل ۴ آمده است و در اینجا تکرار نمی‌شود.

۲.آ خانواده teacher

```
// observable: teacher.TEACH
```

1

reactiveclass Teacher(2) {	2
knownrebecs {	3
Student student;	4
}	5
statevars {	6
int lessonLevel;	7
int wand;	8
}	9
Teacher() {	10
lessonLevel = 0;	11
wand = 0;	12
self.teach();	13
}	14
msgsrv teach() {	15
wand = (wand + 1) % 3;	16
lessonLevel = (lessonLevel + 1) % 2;	17
student.learn(lessonLevel) after(2);	18
}	19
}	20
reactiveclass Student(2) {	21
knownrebecs {	22
Teacher teacher;	23
}	24
Student() {	25
}	26
msgsrv learn(int receivedLevel) {	27
teacher.teach();	28
}	29
}	30
main {	31
Teacher teacher(student):();	32
Student student(teacher):();	33
}	34

برنامهٔ 1: teacher1 wand
.

// observable: teacher.TEACH	1
reactiveclass Teacher(2) {	2
knownrebecs {	3
Student student;	4
}	5
statevars {	6
int lessonLevel;	7
int wand;	8
}	9
Teacher() {	10
lessonLevel = 0;	11
wand = 0;	12
self.teach();	13

کاهش مدل معنایی ربکا با استفاده از رابطه شبیه‌سازی دوسویه ضعیف زمان‌دار پیوست آ: مدل‌های ربکای به‌کاررفته در ارزیابی

}	14
msgsrv teach() {	15
lessonLevel = (lessonLevel + 1) % 2;	16
student.learn(lessonLevel) after(2);	17
}	18
}	19
reactiveclass Student(2) {	20
knownrebecs {	21
Teacher teacher;	22
}	23
Student() {	24
}	25
msgsrv learn(int receivedLevel) {	26
teacher.teach();	27
}	28
}	29
main {	30
Teacher teacher(student):();	31
Student student(teacher):();	32
}	33

برنامه 2: teacher2 wand

.

reactiveclass Teacher(2) {	1
knownrebecs {	2
Student student;	3
}	4
statevars {	5
int lessonLevel;	6
}	7
Teacher() {	8
lessonLevel = 0;	9
self.teach();	10
}	11
msgsrv teach() {	12
lessonLevel = (lessonLevel + 1) % 2;	13
student.learn(lessonLevel) after(2);	14
}	15
}	16
reactiveclass Student(2) {	17
knownrebecs {	18
Teacher teacher;	19
}	20
statevars {	21
int wand;	22
}	23
Student() {	24
wand = 0;	25
}	26

پیوست آ: مدل‌های ربکای به‌کاررفته در ارزیابی کاهش مدل معنایی ربکا با استفاده از رابطه شبیه‌سازی دوسویه ضعیف زمان‌دار

msgsrv learn(int receivedLevel) {	27
wand = (wand + 1) % 3;	28
teacher.teach();	29
}	30
}	31
main {	32
Teacher teacher(student):();	33
Student student(teacher):();	34
}	35

برنامه ۳: wand teacher3

۳.آ خانواده chain

// observable: worker.DECIDE, controller.APPROVED, controller.REJECTED	1
reactiveclass Controller(2) {	2
knownrebecs {	3
Worker worker;	4
}	5
Controller() {	6
self.start();	7
}	8
msgsrv start() {	9
worker.prepare() after(1);	10
}	11
msgsrv forwardDecision() {	12
worker.decide() after(1);	13
}	14
msgsrv approved() {	15
worker.complete() after(1);	16
}	17
msgsrv rejected() {	18
worker.complete() after(1);	19
}	20
msgsrv restart() {	21
worker.prepare() after(1);	22
}	23
}	24
reactiveclass Worker(2) {	25
knownrebecs {	26
Controller controller;	27
}	28
statevars {	29
boolean choice;	30

کاهش مدل معنایی ربکا با استفاده از رابطه شبیه‌سازی دوسویه ضعیف زمان‌دار پیوست آ: مدل‌های ربکای به‌کاررفته در ارزیابی

}	31
Worker() {	32
}	33
msgsrv prepare() {	34
controller.forwardDecision();	35
}	36
msgsrv decide() {	37
choice = ?(true, false);	38
if (choice) {	39
controller.approved() after(3);	40
} else {	41
controller.rejected() after(6);	42
}	43
}	44
msgsrv complete() {	45
controller.restart() after(1);	46
}	47
}	48
main {	49
Controller controller(worker):();	50
Worker worker(controller):();	51
}	52

برنامه 4: chain1

.

// observable: worker.DECIDE, controller.APPROVED, controller.REJECTED	1
reactiveclass Controller(2) {	2
knownrebecs {	3
Worker worker;	4
}	5
Controller() {	6
self.start();	7
}	8
msgsrv start() {	9
worker.decide() after(2);	10
}	11
msgsrv approved() {	12
worker.decide() after(4);	13
}	14
msgsrv rejected() {	15
worker.decide() after(4);	16
}	17
}	18
reactiveclass Worker(2) {	19
knownrebecs {	20
Controller controller;	21
}	22
statevars {	23
boolean choice;	24

پیوست آ: مدل‌های ربکای به‌کاررفته در ارزیابی کاهش مدل معنایی ربکا با استفاده از رابطه شبیه‌سازی دوسویه ضعیف زمان‌دار

}	25
Worker() {	26
}	27
msgsrv decide() {	28
choice = ?(true, false);	29
if (choice) {	30
controller.approved() after(3);	31
} else {	32
controller.rejected() after(6);	33
}	34
}	35
}	36
main {	37
Controller controller(worker):();	38
Worker worker(controller):();	39
}	40

برنامه 5: chain2 .

کتاب نامه

- [1] Alur, Rajeev and Dill, David L. Automata for modeling real-time systems. In *Automata, Languages and Programming*, volume 443 of *Lecture Notes in Computer Science*, pages 322–335. Springer, 1990.
- [2] Alur, Rajeev and Dill, David L. A theory of timed automata. *Theoretical Computer Science*, 126(2):183–235, 1994.
- [3] Baier, Christel and Katoen, Joost-Pieter. *Principles of Model Checking*. MIT Press, Cambridge, MA, USA, 2008.
- [4] Bengtsson, Johan and Yi, Wang. Timed automata: Semantics, algorithms and tools. In *Lectures on Concurrency and Petri Nets*, volume 3098 of *Lecture Notes in Computer Science*, pages 87–124. Springer, 2004.
- [5] Bérard, Béatrice, Cassez, Franck, Haddad, Serge, Lime, Didier, and Roux, Olivier H. When are timed automata weakly timed bisimilar to time Petri nets? *Theoretical Computer Science*, 403(2–3):202–220, 2008.
- [6] Čerāns, Kārlis. Decidability of bisimulation equivalences for parallel timer processes. In *Computer Aided Verification*, volume 663 of *Lecture Notes in Computer Science*, pages 302–315. Springer, 1993.
- [7] Clarke, Edmund M., Grumberg, Orna, and Peled, Doron A. *Model Checking*. MIT Press, Cambridge, MA, USA, 1999.
- [8] European Space Agency. Ariane 501: Presentation of inquiry board report. https://www.esa.int/Newsroom/Press_Releases/Ariane_501_-_Presentation_of_Inquiry_Board_report, 1996. Published 23 July 1996; accessed 9 September 2026.
- [9] Groote, Jan Friso, Jansen, David N., Keiren, Jeroen J. A., and Wijs, Anton J. An $o(m \log n)$ algorithm for computing stuttering equivalence and branching bisimulation. *ACM Transactions on Computational Logic*, 18(2):13:1–13:34, 2017.

- [10] Groote, Jan Friso and Vaandrager, Frits W. An efficient algorithm for branching bisimulation and stuttering equivalence. In *Automata, Languages and Programming (ICALP 1990)*, volume 443 of *Lecture Notes in Computer Science*, pages 626–638. Springer, 1990.
- [11] Henzinger, Thomas A., Nicollin, Xavier, Sifakis, Joseph, and Yovine, Sergio. Symbolic model checking for real-time systems. *Information and Computation*, 111(2):193–244, 1994.
- [12] Lieb, Alexander, Göttmann, Hendrik, Lochau, Malte, Luthmann, Lars, and Schürr, Andy. Checking timed bisimilarity with virtual clocks. *Fundamenta Informaticae*, 185(1), 2025. Article 2102; arXiv:2412.15799.
- [13] Meta Engineering. Open-sourcing facebook infer: Identify bugs before you ship. <https://engineering.fb.com/2015/06/11/developer-tools/open-sourcing-facebook-infer-identify-bugs-before-you-ship/>, 2015. Published 11 June 2015; accessed 9 September 2026.
- [14] Milner, Robin. *Communication and Concurrency*. Prentice Hall International Series in Computer Science. Prentice Hall, 1989.
- [15] Mobed, Amirparsa. Afra weak timed reduction. <https://github.com/ParsaMOBB/AfraWeakTimedReduction>, 2026. Version 1.0.0; thesis snapshot a344925 and recorded evaluation revision 3d63904; accessed 9 September 2026.
- [16] Newcombe, Chris, Rath, Tim, Zhang, Fan, Munteanu, Bogdan, Brooker, Marc, and Deardeuff, Michael. How amazon web services uses formal methods. *Communications of the ACM*, 58(4):66–73, 2015.
- [17] Paige, Robert and Tarjan, Robert E. Three partition refinement algorithms. *SIAM Journal on Computing*, 16(6):973–989, 1987.
- [18] Rebeca Research Group. Afra: An integrated environment for modeling and verifying rebeca family designs. <https://rebeca-lang.org/alltools/Afra>, 2026. Accessed 9 September 2026.
- [19] Rebeca Research Group. Afra source repository. <https://github.com/rebeca-lang/org.rebecalang.afra>, 2026. Source contract examined at commit ed3caf6; accessed 9 September 2026.
- [20] Rebeca Research Group. Rebeca compiler source repository. <https://github.com/rebeca-lang/org.rebecalang.compiler>, 2026. Toolchain configuration examined at commit dc663e0; accessed 9 September 2026.

- [21] Rebeca Research Group. Rebeca model checker source repository. <https://github.com/rebeca-lang/org.rebecalang.rmc>, 2026. TTS emitter examined at commit cab60b1; accessed 9 September 2026.
- [22] Rebeca Research Group. Rebeca model checker source repository. <https://github.com/rebeca-lang/org.rebecalang.modelchecker>, 2026. Toolchain configuration examined at commit fb4197f; accessed 9 September 2026.
- [23] Rebeca Research Group. Rebeca modeling language: Official github organization. <https://github.com/rebeca-lang>, 2026. Accessed 9 September 2026.
- [24] Rebeca Research Group. Rebeca state-space analysis source repository. <https://github.com/rebeca-lang/org.rebecalang.statespaceanalysis>, 2026. State-space reader examined at commit b2b0e23; accessed 9 September 2026.
- [25] Rebeca Research Group. Rebeca state-space transformer source repository. <https://github.com/rebeca-lang/org.rebecalang.statespacetransformer>, 2026. State-space reader examined at commit 02fbef1; accessed 9 September 2026.
- [26] Reynier, Pierre-Alain and Sangnier, Arnaud. Weak time petri nets strike back! In *CONCUR 2009 – Concurrency Theory*, volume 5710 of *Lecture Notes in Computer Science*, pages 557–571. Springer, 2009.
- [27] Reynisson, Arni Hermann, Sirjani, Marjan, Aceto, Luca, Cimini, Matteo, Jafari, Ali, Ingólfssdóttir, Anna, and Sigurdarson, Steinar Hugi. Modelling and simulation of asynchronous real-time systems using timed rebeca. *Science of Computer Programming*, 89:41–68, 2014.
- [28] Sirjani, Marjan. *Formal Specification and Verification of Concurrent and Reactive Systems*. PhD thesis, Sharif University of Technology, Tehran, Iran, June 2004.
- [29] Sirjani, Marjan, Movaghar, Ali, Shali, Amin, and de Boer, Frank S. Modeling and verification of reactive systems using rebeca. *Fundamenta Informaticae*, 63(4):385–410, 2004.
- [30] Swamy, Nikhil. Project everest: Advancing the science of program proof. Microsoft Research, <https://www.microsoft.com/en-us/research/blog/project-everest-advancing-the-science-of-program-proof/>, 2019. Published 5 September 2019; accessed 9 September 2026.
- [31] United States General Accounting Office. Patriot missile defense: Software problem led to system failure at dhahran, saudi arabia. Technical Report IMTEC-92-26, United States General Accounting Office, 1992. <https://www.gao.gov/products/imtec-92-26>.

کاهش مدل معنایی ربکا با استفاده از رابطه شبیه‌سازی دوسویه ضعیف زمان‌دار پیوست‌آ: مدل‌های ربکای به‌کاررفته در ارزیابی

- [32] van Glabbeek, Rob J. and Weijland, W. Peter. Branching time and abstraction in bisimulation semantics. *Journal of the ACM*, 43(3):555–600, 1996.

واژه‌نامه فارسی به انگلیسی

الف ربکای زمان‌دار Timed Rebeca

رد Trace آزمون انتها به انتها End-to-End Testing

روش صوری Formal Method افرا Afra

افراز Partition

انفجار فضای حالت .. State-Space Explosion

س

سامانه انتقال برچسب‌دار ... Labeled Transition System پ

سامانه انتقال خارج قسمتی . Quotient Transition System پالایش افراز Partition Refinement

پالایش صوری Formal Refinement

سامانه انتقال زمان‌دار Timed Transition System پنهان‌سازی Hiding

سیستم بلادرنگ Real-Time System

سیستم واکنشی Reactive System ت

توسعه صوری Formal Development

ش

شبیه‌سازی دوسویه Bisimulation د

شبیه‌سازی دوسویه انشعابی Branching Bisimulation درستی‌یابی صوری Formal Verification

درستی‌یابی نرم‌افزار Software Verification

شبیه‌سازی دوسویه ضعیف . Weak Bisimulation

ر

ک

رابط خط فرمان ... Command-Line Interface

کنش داخلی Internal Action ربکا Rebeca

سوم

م

مدل کنشگر Actor Model

مشخصه‌نویسی صوری . . Formal Specification

و

وارسی مدل Model Checking

واژه‌نامه انگلیسی به فارسی

H

Hiding پنهان‌سازی

I

Internal Action کنش داخلی

L

Labeled Transition سامانه انتقال برچسب‌دار

M

Model Checking واریسی مدل

P

Partition افراز

Partition Refinement پالایش افراز

Q

Quotient Transition سامانه انتقال خارج‌قسمتی

A

Actor Model مدل کنشگر

Afra افرا

B

Bisimulation شبیه‌سازی دوسویه

Branching شبیه‌سازی دوسویه انشعابی

Bisimulation

C

Command-Line Interface رابط خط فرمان

E

End-to-End Testing آزمون انتها‌به‌انتها

F

Formal Development توسعه صوری

Formal Verification درستی‌یابی صوری

Formal Method روش صوری

Formal Refinement پالایش صوری

Formal Specification مشخصه‌نویسی صوری

System

R

Reactive System سیستم واکنشی
Real-Time System سیستم بلادرنگ
Rebeca ربکا

S

Software Verification درستی‌یابی نرم‌افزار
State-Space Explosion . . انفجار فضای حالت

T

Timed Rebeca ربکای زمان‌دار
Timed Transition System . سامانه انتقال زمان‌دار
Trace رد

W

Weak Bisimulation . شبیه‌سازی دوسویه ضعیف

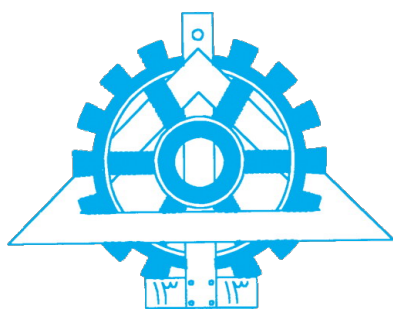
Abstract

Model checking automatically verifies desired properties by exploring a system's state space. In concurrent real-time models, the combination of component schedules, message queues and timing constraints leads to state-space explosion, and what that explosion produces is a very large timed transition system. This thesis does not attempt to prevent that explosion while the state space is being generated. Its problem is to make the resulting system smaller *after* it has been generated, so that it becomes practical to analyse, to inspect, and to compare against another model.

To that end, a tool was built that reads Afra's timed state-space export, hides the interactions the user did not declare observable, and groups the states that cannot be distinguished by observable behaviour and elapsed time. It then builds a smaller quotient model from those classes and checks its relation to the input with an independent verifier. The equivalence criterion is weak timed bisimilarity, and the structure of the algorithm is adapted from the partition refinement proposed for computing branching bisimulation on labelled transition systems.

The Rebeca and Afra toolchain generates and displays timed state spaces, but no tool was previously available to its users that reduced such a state space with respect to a chosen set of observable interactions; providing one is the main contribution of this work. Evaluation on the available examples and on state spaces generated from real Rebeca models shows that the method reduces model size while preserving the selected behavioural criterion, and that different models with the same observable behaviour are carried to the same reduced model.

Keywords Rebeca, Afra, Weak Timed Bisimulation, Formal Methods, Model Checking



University of Tehran
College of Engineering
School of Electrical and
Computer Engineering
Department of Computer
Engineering



Reduction of Rebeca Semantic Models Using Weak Timed Bisimulation

Bachelor of Science Thesis in Computer Engineering

By:

Amirparsa Mobed

Supervisor:

Dr. Fatemeh Ghassemi Esfahani

September 2026